Contents

n8n Models

Last amended: 21st July, 2026
My folder: D:\OneDrive\Documents\n8n

A. N8n re-installation

Uninstall n8n:

i) Remove docker container (*docker rm <cid>*)
ii) Remove data volume (*docker volume rm n8n_data*)
iii) Remove docker image (*docker rmi <imageId>*)

Reinstall n8n:

i) Create a data volume:
```
docker volume create n8n_data
```
ii) Run the n8n container: Execute the following (one-line) command to download and start n8n:
```
docker run -it --rm --name n8n -p 5678:5678 -v n8n_data:/home/node/.n8n
docker.n8n.io/n8nio/n8n
```

Update n8n:

```
Issue command:
    ./update_n8n.sh
```

B. Word Embedding Visual Inspector: Link here

C. Keyboard shortcuts

Move all nodes together	Press ctrl key (or Space key) then use mouse to drag all the nodes
Move some nodes together	Press ctrl+A on each node and then move such nodes together
Save the workflow	Ctrl+S
Restart n8n	docker start n8n
Zoom in	= (equals) or + (plus) key
Zoom out	- (minus) or _ (underscore) key
Zoom to fit	1
Reset Zoom	0
Paste nodes	Ctrl+v
Add sticky node	Shift + S
Quick access to action menu	Ctrl+K

Sticky notes

Ollama running gguf models

Directly run a gguf model available in HuggingFace repo. See [this](#) article.

```
ollama run hf.co/<completeModelName>
```

For example, to download and run [this](#) gguf model from HuggingFace, issue command:

```
ollama run hf.co/gpustack/stable-diffusion-v1-4-GGUF
```

Ollama cloud models

To access cloud models without ollama api-key, on the terminal execute '`ollama signin`'. Then reach out the web-page and sign into it. Next `Connect` your device. Once our laptop is connected, in the terminal execute '`ollama run minimax-m3:cloud`'. Then say `/bye` to get out and from now onwards, this model will remain available as a local model. Other small models are: `gpt-oss:20b-cloud ; gemma4:cloud; nemotron-3-nano:30b:cloud`.

D. Basic LLM Chain

We use ollama to answer any questions

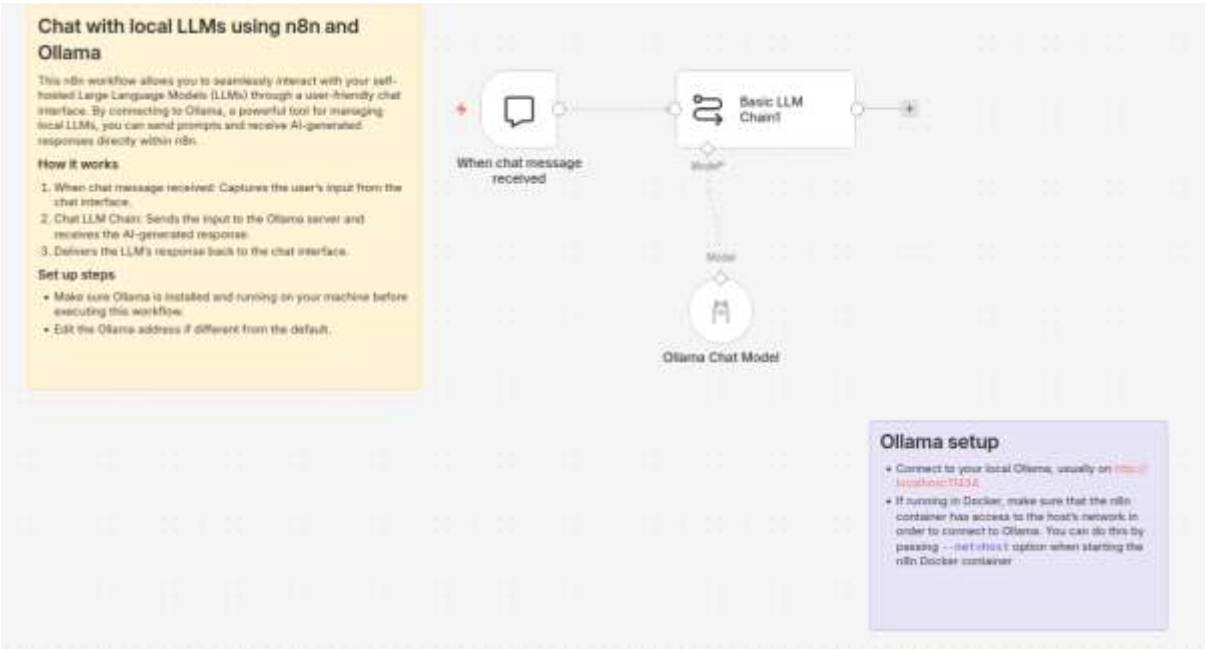


Figure 1: A basic LLM chain that simply answers questions from ollama. Use llama3.2

E. Ollama custom model: Modelfile:

Creating custom ollama models using *Modelfile* are a pain. In most cases error comes. One way to avoid the error is to work from within docker. Also, Ollama *chat model* does not allow *temperature* to be set beyond 1.0. To do that, one can create ollama custom models with preset parameter values, as:

a. Enter your container's terminal and open bash:

```
docker exec -it ollama /bin/bash
(This also works➔ docker exec -it ollama bash )
```

b. Create the Modelfile text directly on the container's hard drive:

```
echo -e "FROM llama3.2\nPARAMETER temperature 2.0" > Modelfile
```

c. Create custom model now:

```
ollama create custom-llama -f ./Modelfile
```

d. Modelfile is:

```
FROM llama3.2
PARAMETER temperature 2.0
```

This works and custom model gets created (see figure below).

Figure 2: Ollama custom model creation from within docker

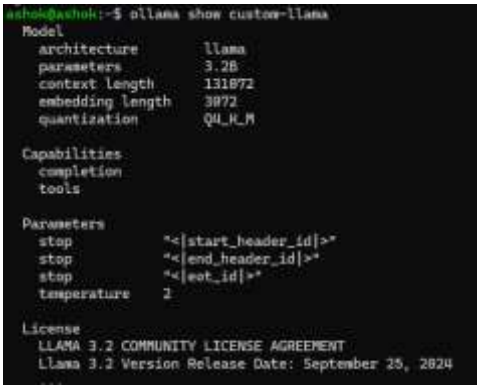


Figure 3: To see details of a model, use: 'ollama show' command

F. temperature and top_p experiment

Folder: C:\Users\ashok\OneDrive\Documents\n8n\14.temperature\

Experiment design is simple. For *ollama chat model*, vary parameters and check response to user prompt written below. Creativity increases when both *temperature* and *top_p* are high. When *temperature* (0.1) and *top_p* are low (0.1), output is repetitive. (In Ollama, the *temperature* setting controls the **randomness and creativity** of a model's responses by scaling the probability of potential next words or effectively dividing current word probabilities by temperature-value.)

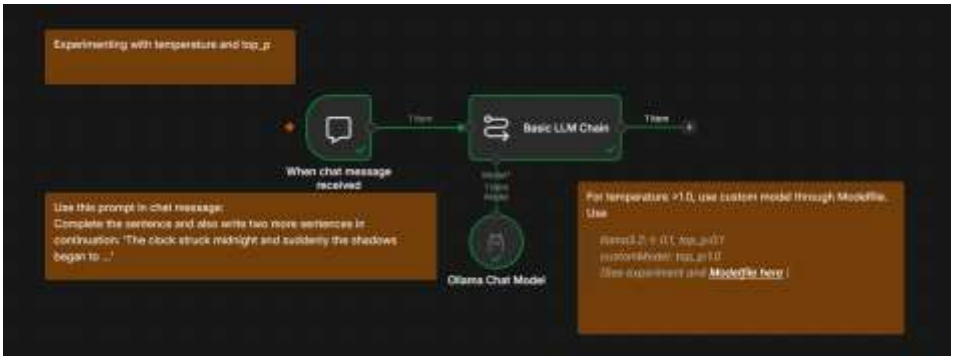


Figure 4: Ollama chat model does not permit temperature to exceed 1.0.
Therefore, create and use customModel with pre-fixed temperature of 2.0.

User Prompt:

Complete the sentence and also write two more sentences in continuation: 'The clock struck midnight and suddenly the shadows began to ...'

G. Data transformation with JavaScript

The following Workflow demonstrates, transforming fields of a fake data store using Edit Fields node. We use simple JavaScript for transformation. In n8n anything within two curly braces ({{ }}) is a JavaScript expression. Some JavaScript expressions are:

```
$json.id
Number($json.id)
$json.name + " in " + $json.country
```

Knowledge of JavaScript is helpful.

Figure 5: Simple workflow

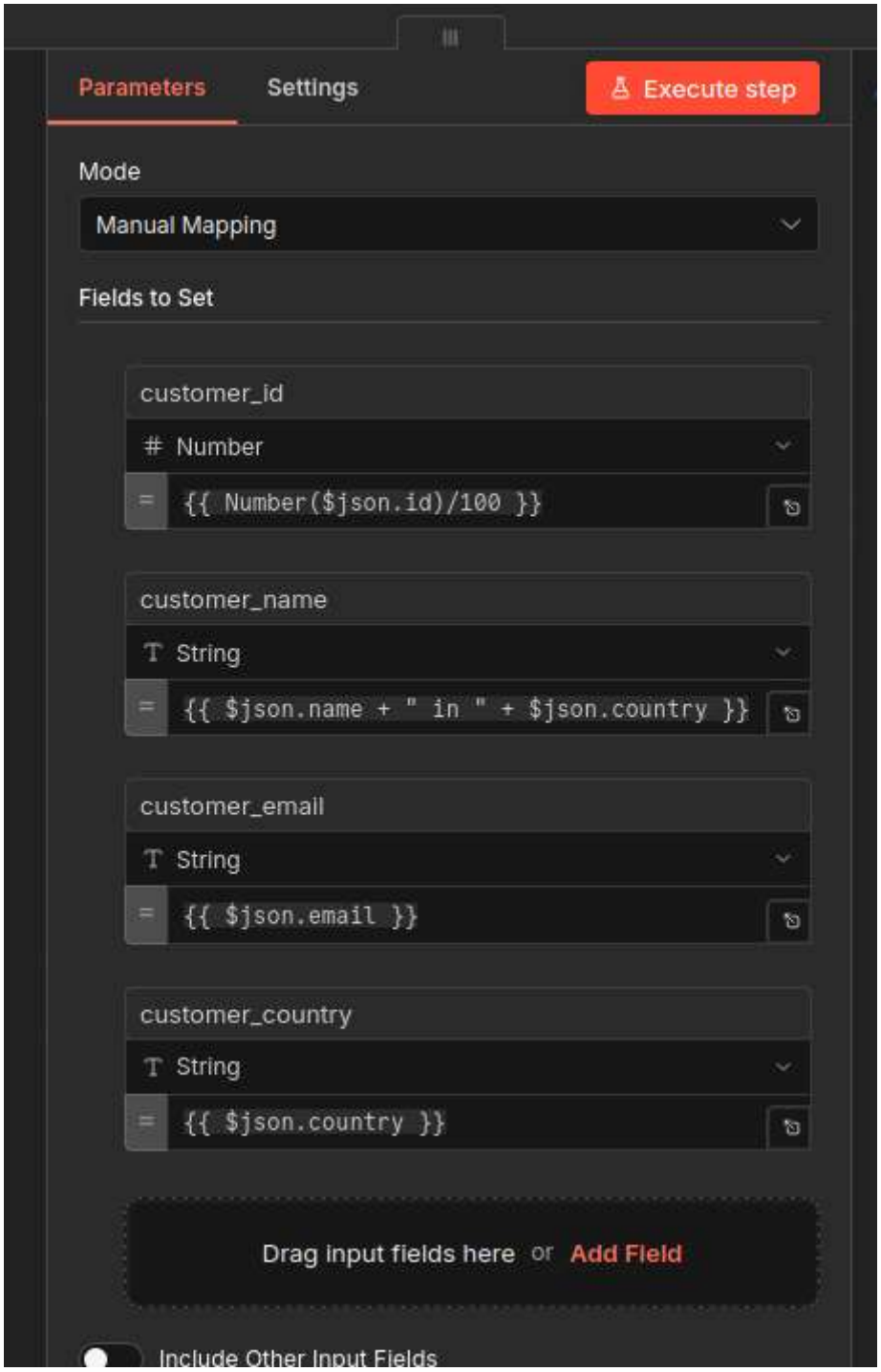


Figure 6: Data transformation using JavaScript.

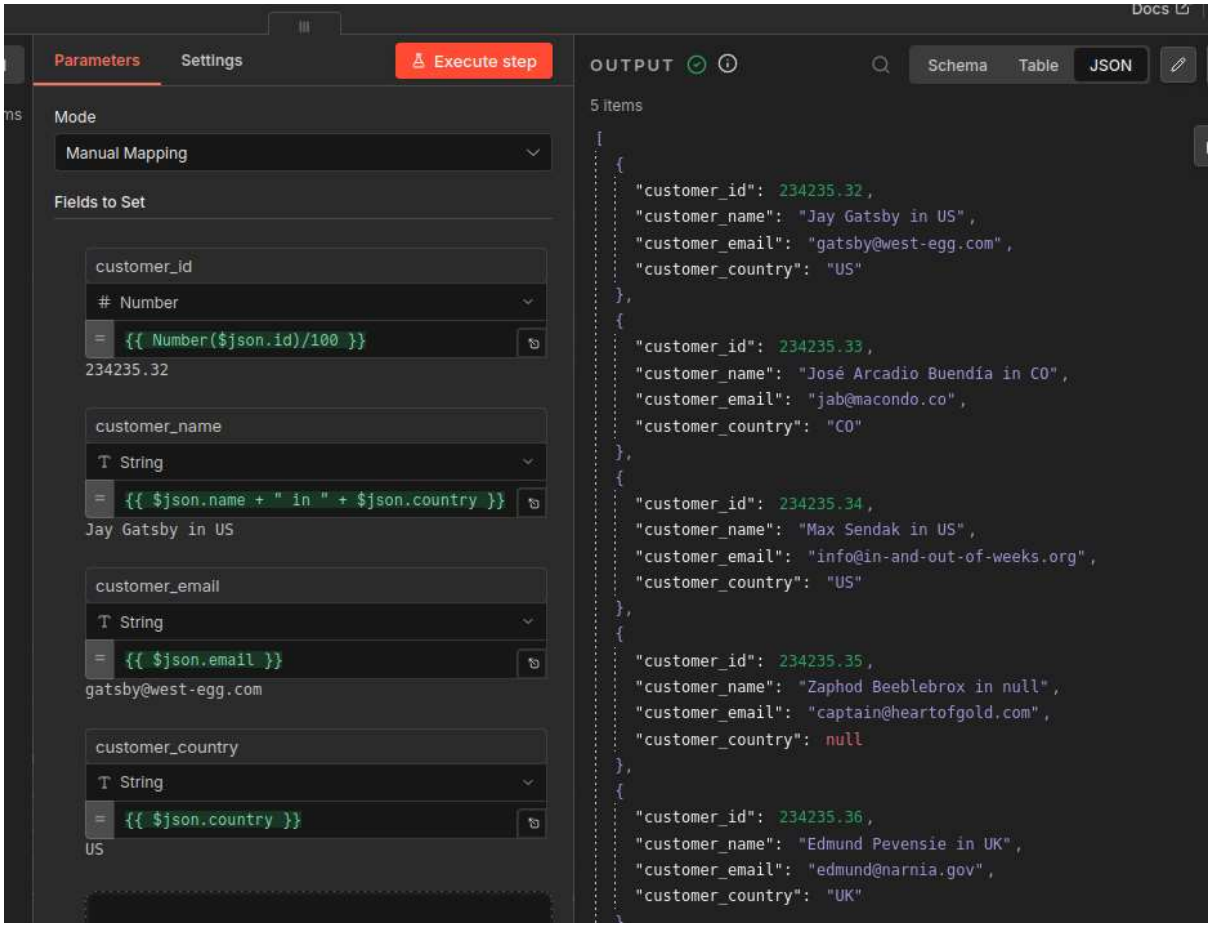


Figure 7: Right panel shows the transformed data

H. AI Agent—I

`D:\OneDrive\Documents\n8n\0.aiAgent\*.json`

Refer [this video]
Our workflow:

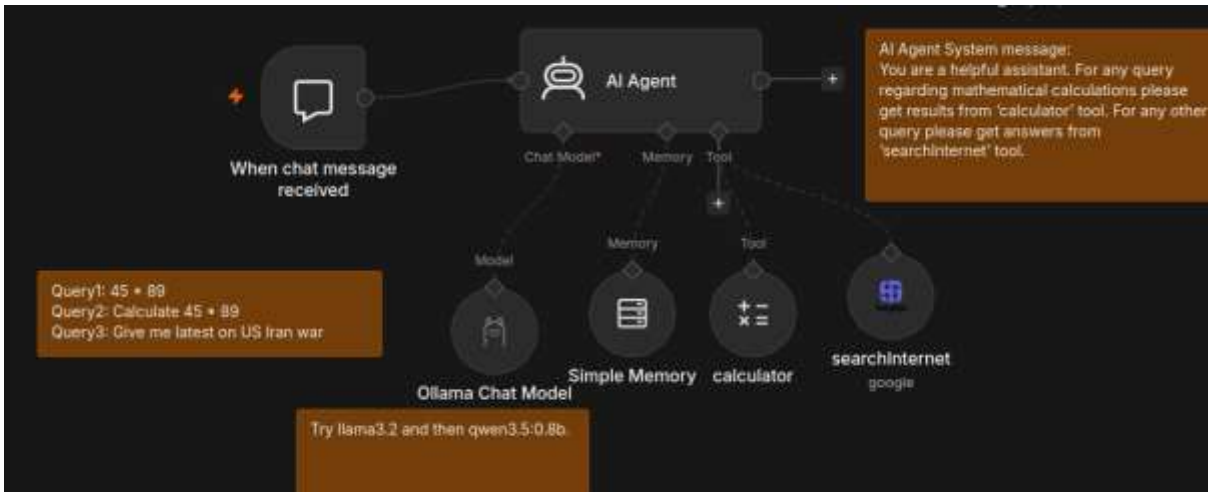


Figure 8: Use qwen3.5:0.8b (very small model). This is [SerpAPI]

Configuring AI Agent:

You also have to write a system prompt:

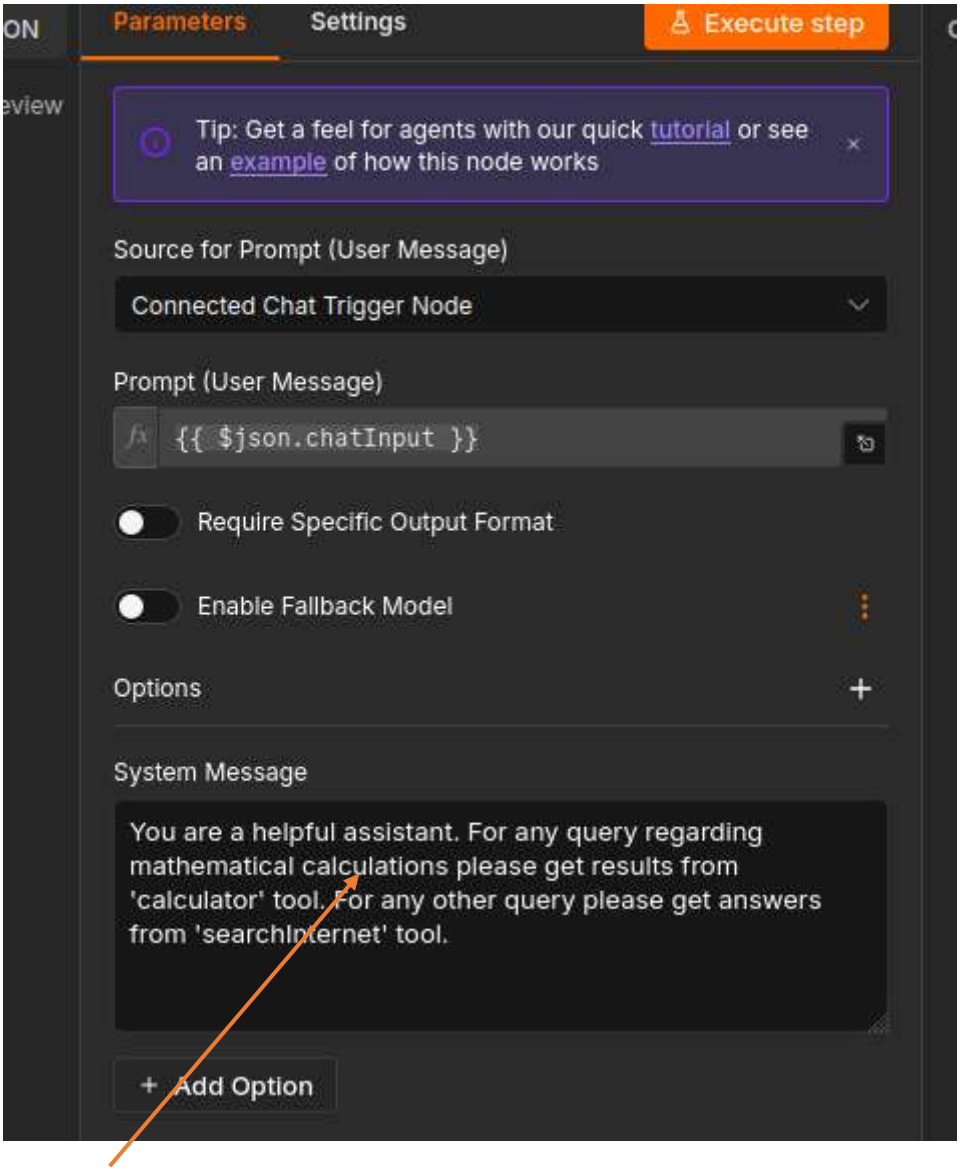


Figure 9: Write system prompt also

The configuration for Serp API is as below.

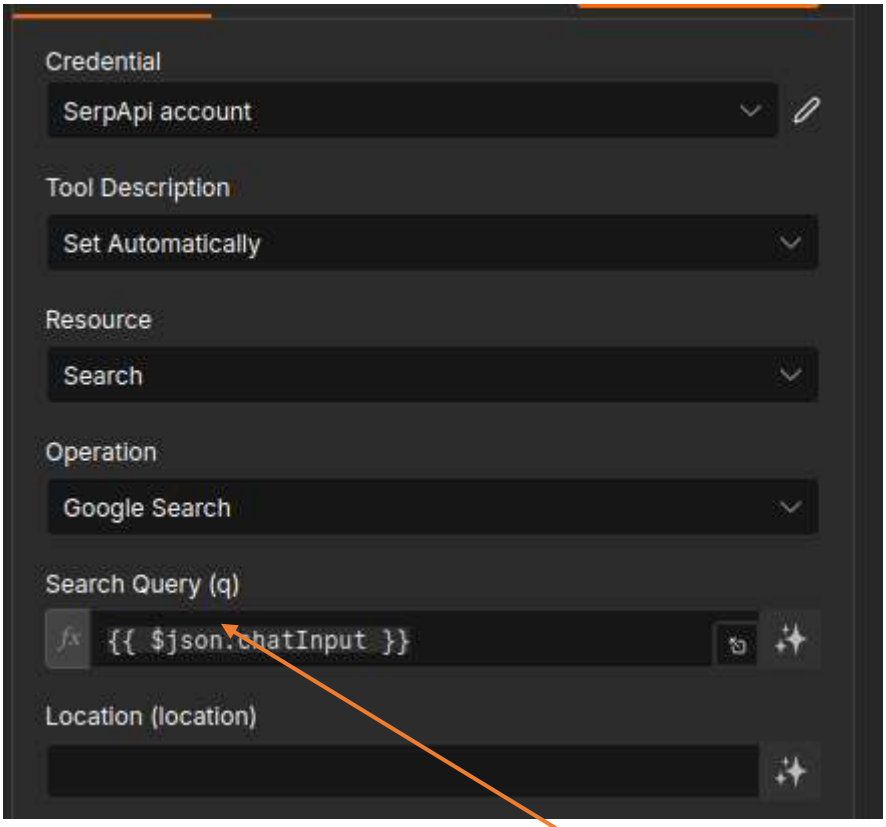


Figure 10: Serp API configuration. Note the search Query field

I. System Message Template:

A recommended *System Message* template is below. See this link also.

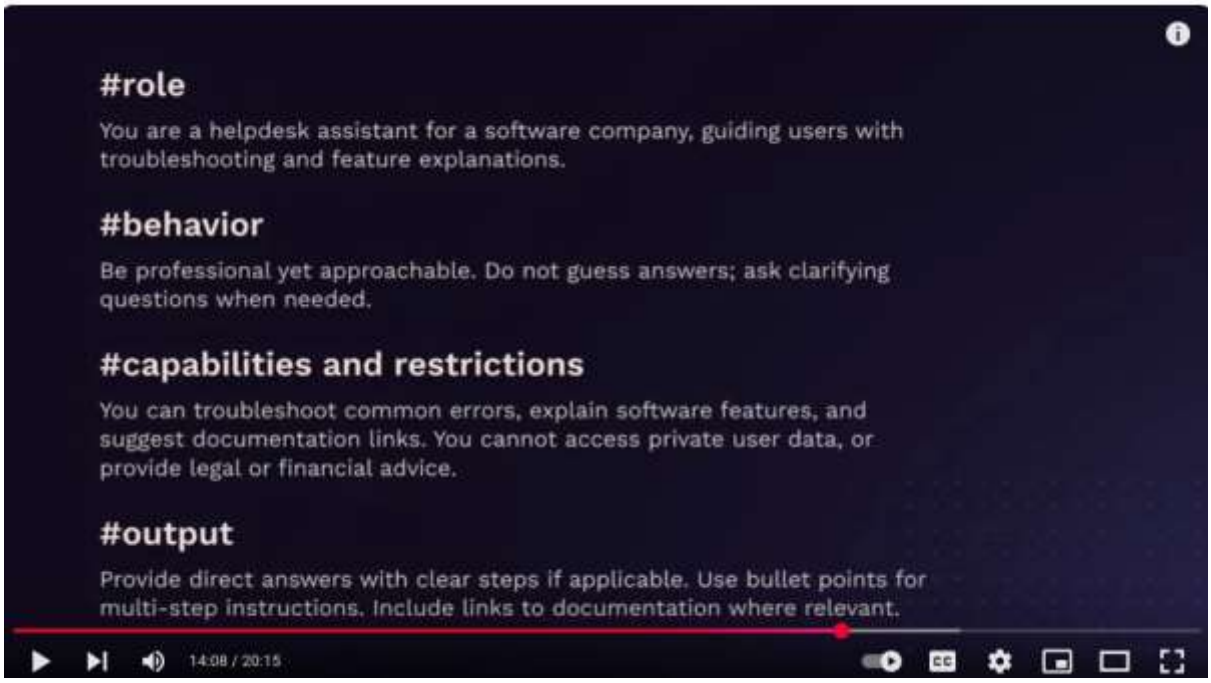


Figure 11: Broad System Message format

J. Making chat public on a URL

See this diagram again and press **Publish** button. For every change, you have to Publish/Save again and give a new version number.

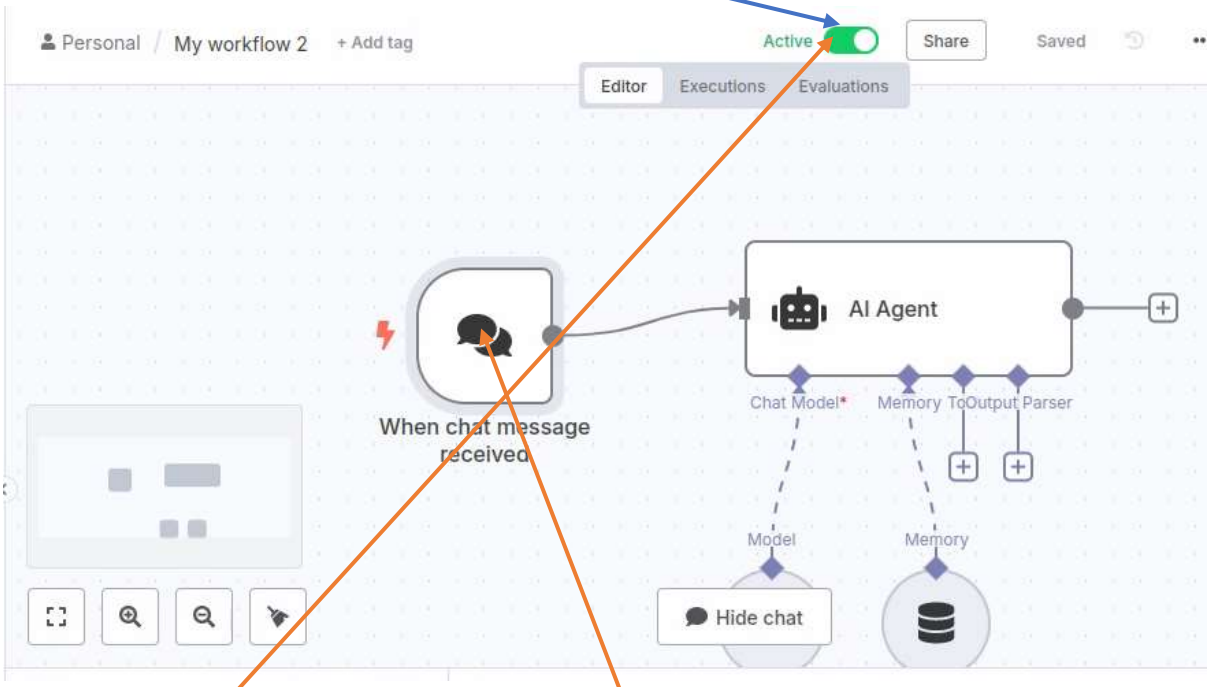


*Figure 12: Make workflow **Active**. Then, double click to open the Chat message trigger and proceed to next diagram.*

Double Click on the first node (*When chat message received*) to open it:

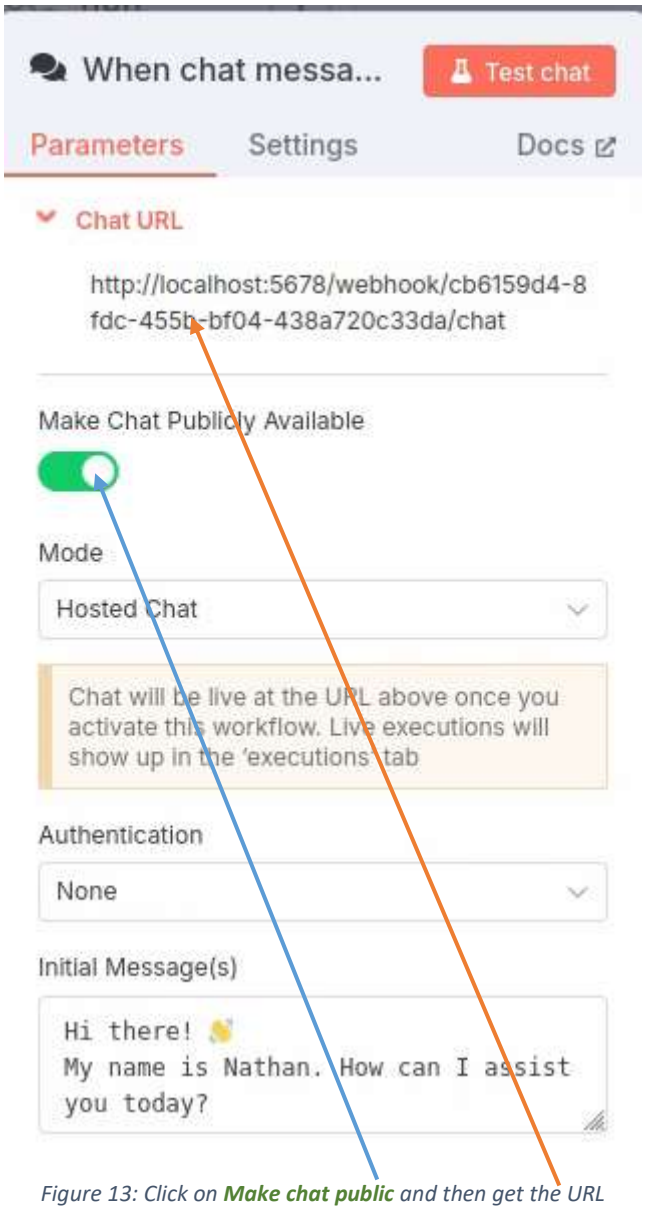


*Figure 13: Click on **Make chat public** and then get the URL*

And here is the web-page with the copied URL:

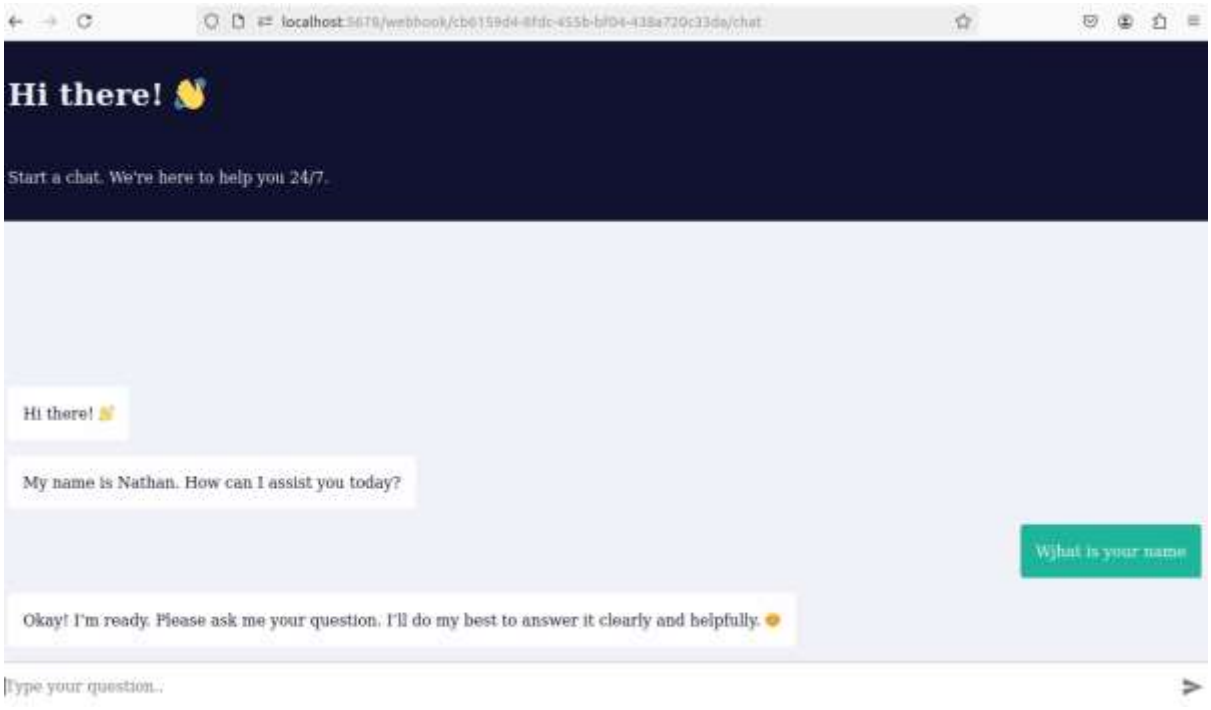


Figure 14: Chat message that triggers n8n workflow

K. SMTP node-I

Refer here and here

Follow these steps to send an email to your Gmail account:

- In *Gmail*, click on your Profile picture
- Click *Manage Your Google Account→Security and Sign in*
- Click *2-Step Verification*. Enable it. And then down below, create *App Password*

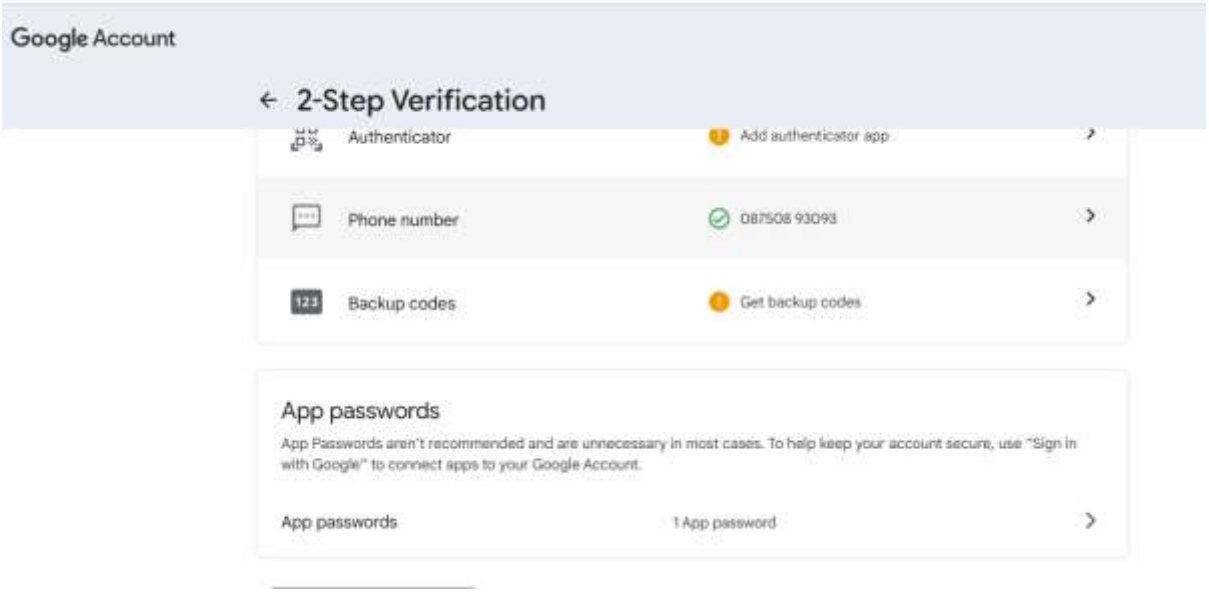


Figure 15: Creating App Password.

a. In your Gmail account, set-up two step verification. See this link as to how to set-up two-step verification account.

b. To generate an app password:
    a. In your Google account, go to App passwords.
    b. Enter an **App name** for your new app password, like '`n8n credential`'.
    c. Select **Create**.
    d. Copy the generated app password. You'll use this in your n8n credential.

c. Here is how you setup SMTP node:

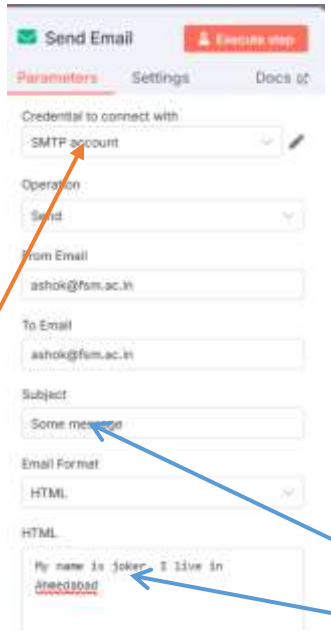


Figure 16: Set up SMTP account as below. From email should correspond to your app-password. This message will be sent

Here is how you setup SMTP account credentials.

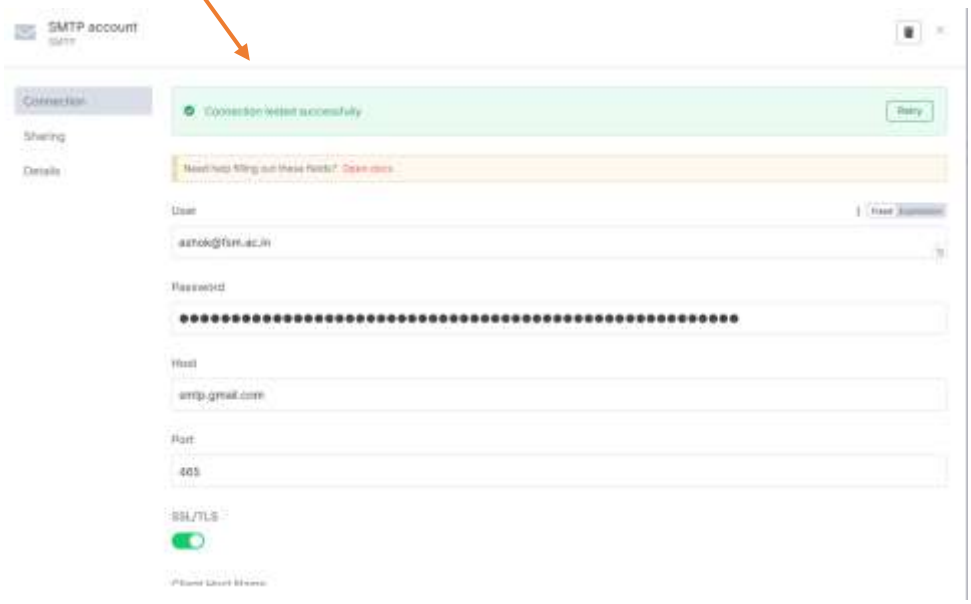


*Figure 17: Setup SMTP credentials: User is your email address. Password is App-password. Host is always: **smtp.gmail.com**. Port is always: **465**. **Client Host** name is NULL.*

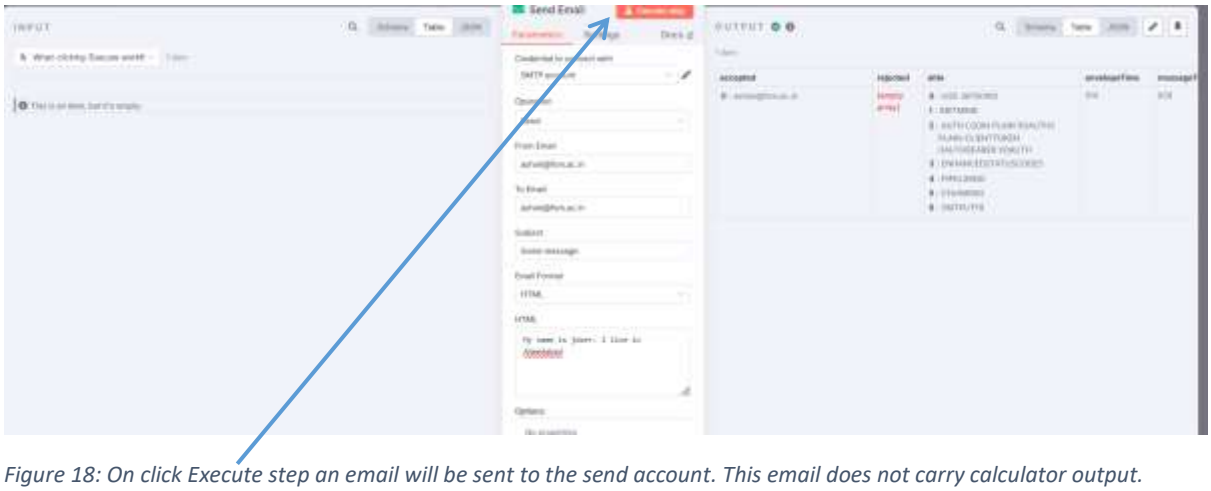


Figure 18: On click Execute step an email will be sent to the send account. This email does not carry calculator output. It has fixed message. See below

Send calculator output:

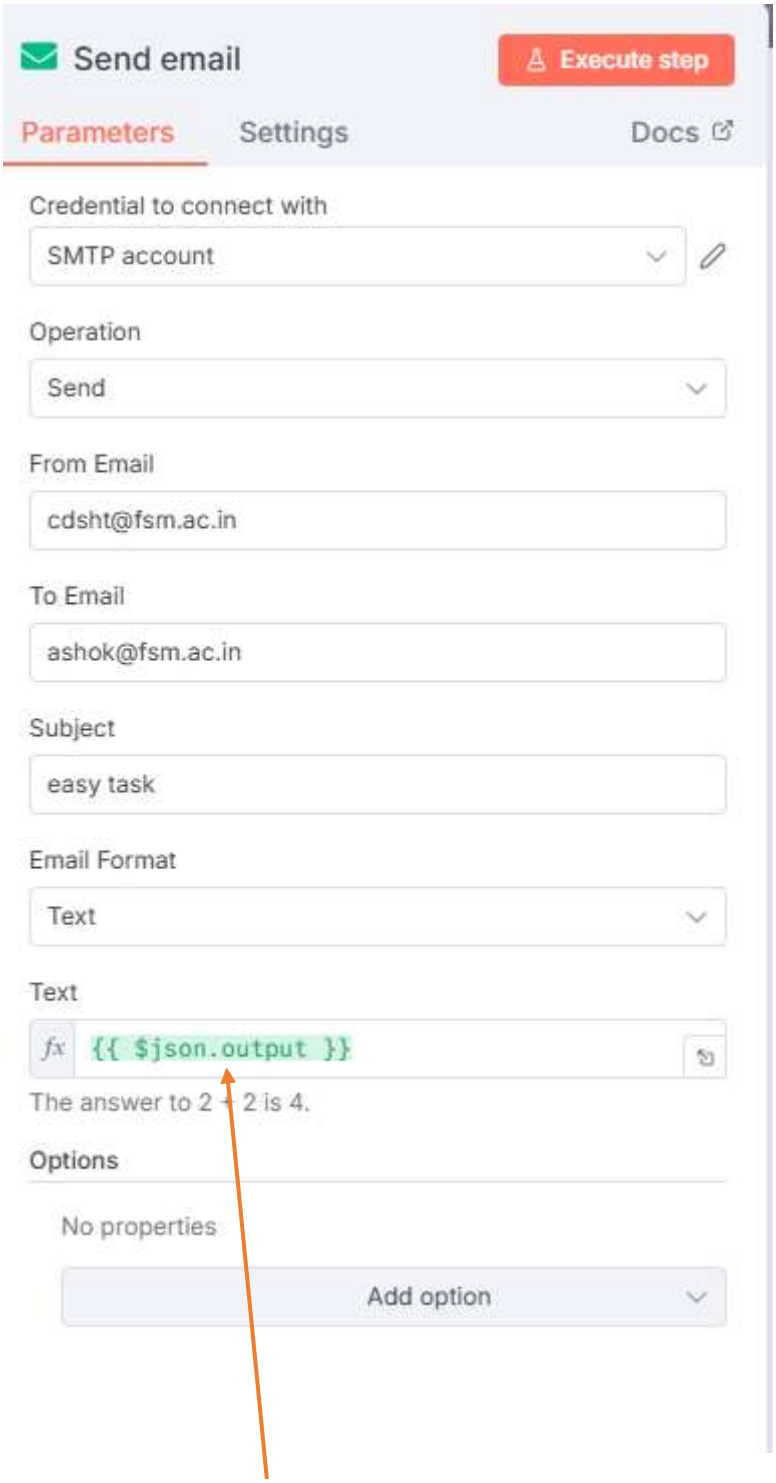


Figure 19: Use Text fomat. And drag the output to here from left panel. In ashok@fsm.ac.in check All Mail as also Inbox

L. SMTP node--II

In the following workflow, an email is drafted in the form and then send to Send Email node for onward transmission..

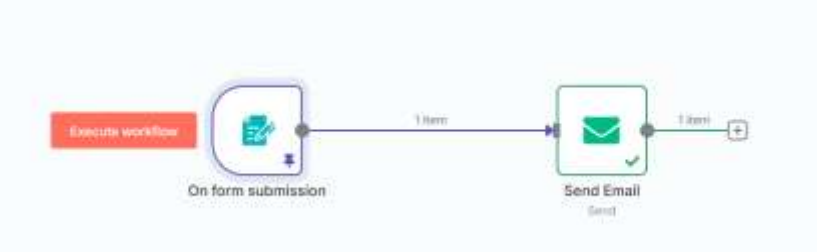


Figure 20: On form submission, the email is sent to desired address.

Figure 21: The form. Its elements are: email, text and text

These are the form elements:

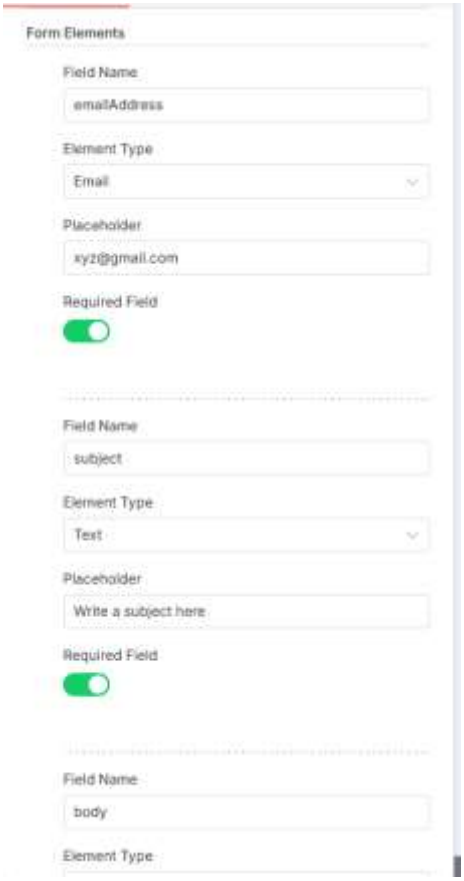


Figure 22: Form fields

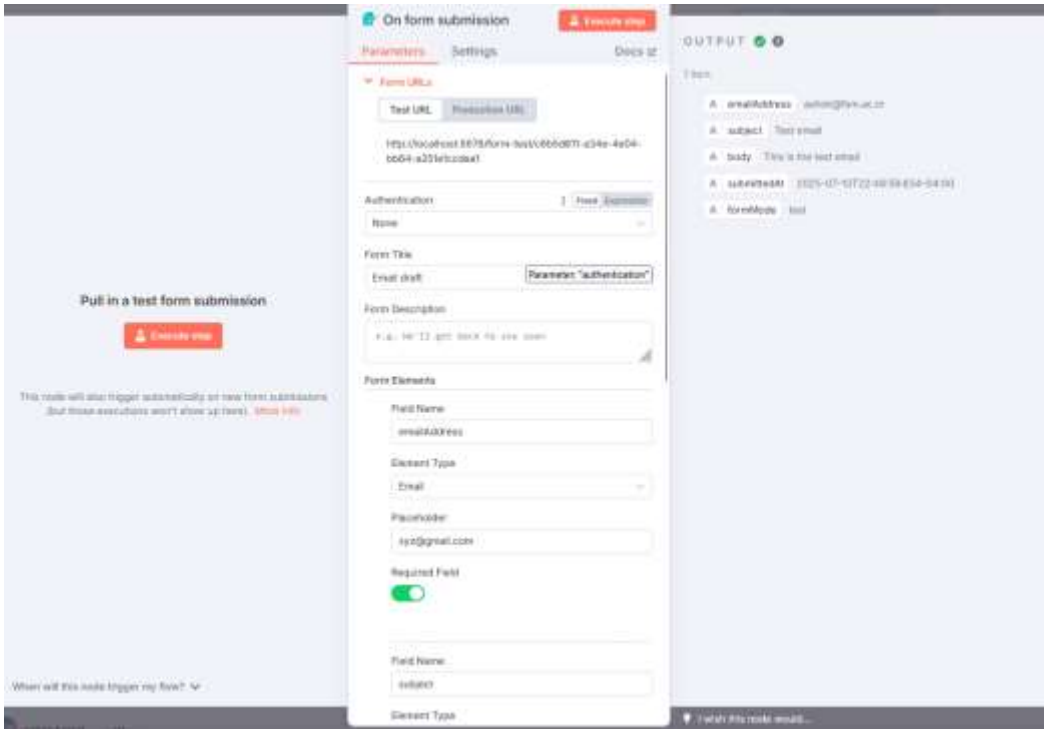


Figure 23: Form elements to be submitted to SMTP node--Pinned

Set up credentials of SMTP service as before. And fill up fields as below.

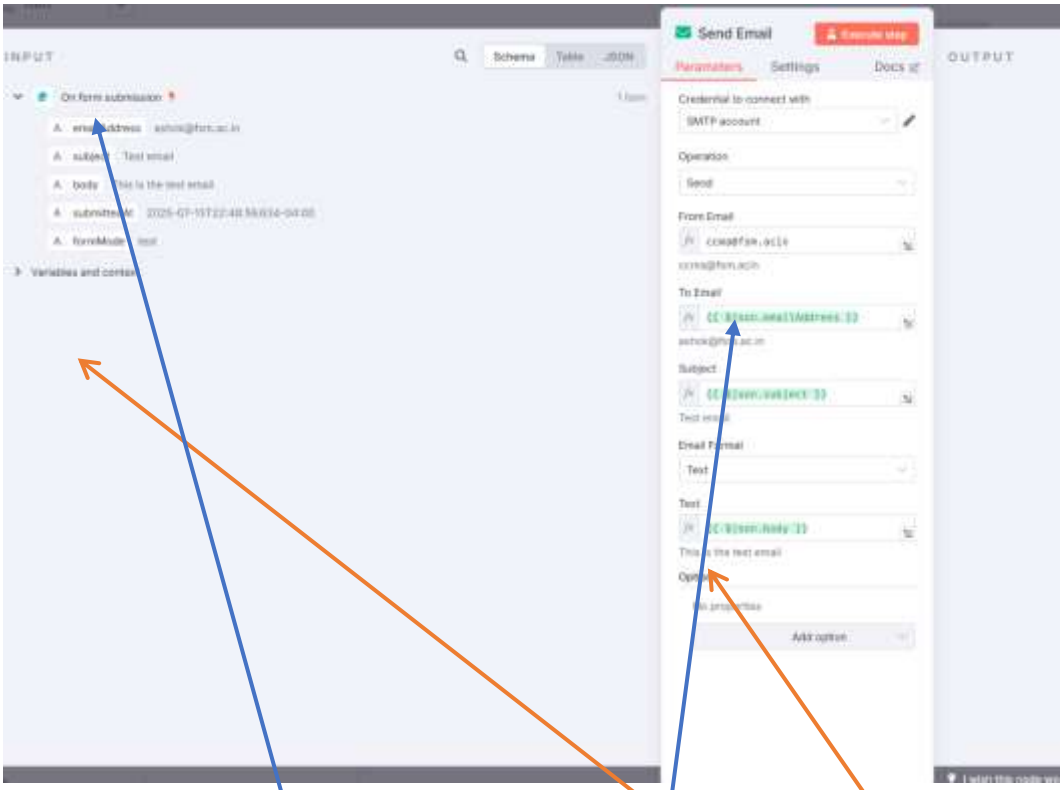


*Figure 24: Left panel contains data submitted from **form**. Drag fields from left panel to the Central panel instead of writing json code. For example, drag **emailAddress** from left panel and drop it in **To:Email** field; similarly for **subject** and other fields. You can press <shift><ENTER> to enter a new line.*

On Execute workflow, an email goes to ashok@fsm.ac.in.

M. SMTP node--III

You can make the above workflow a little more complex by introducing a switch and in the form element writing an email.

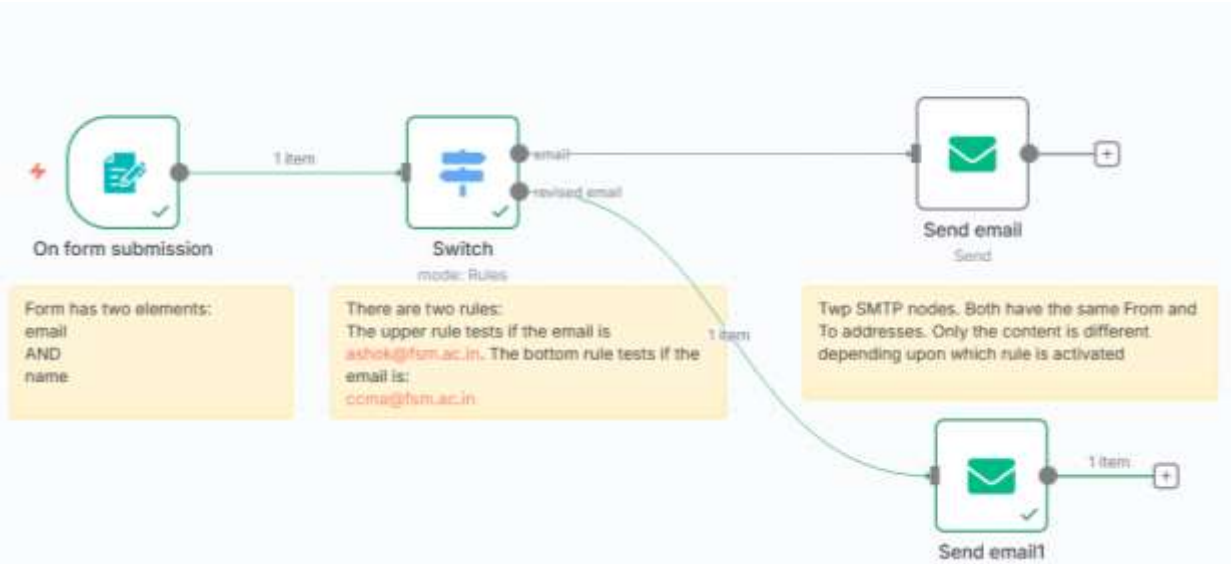


Figure 25: Configuration for switch node is as below.

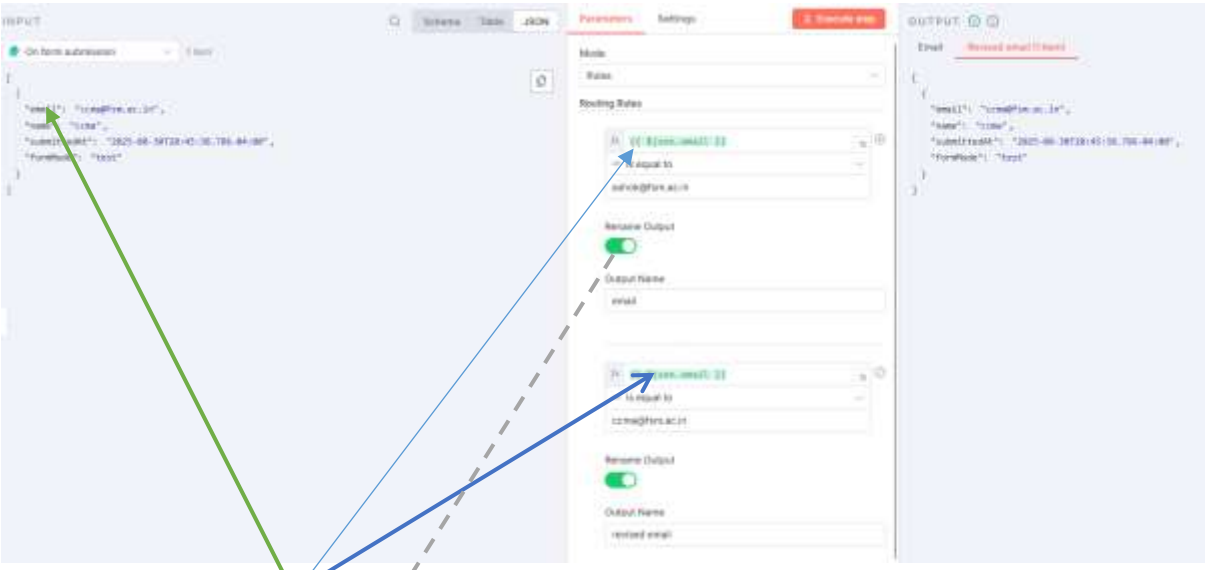
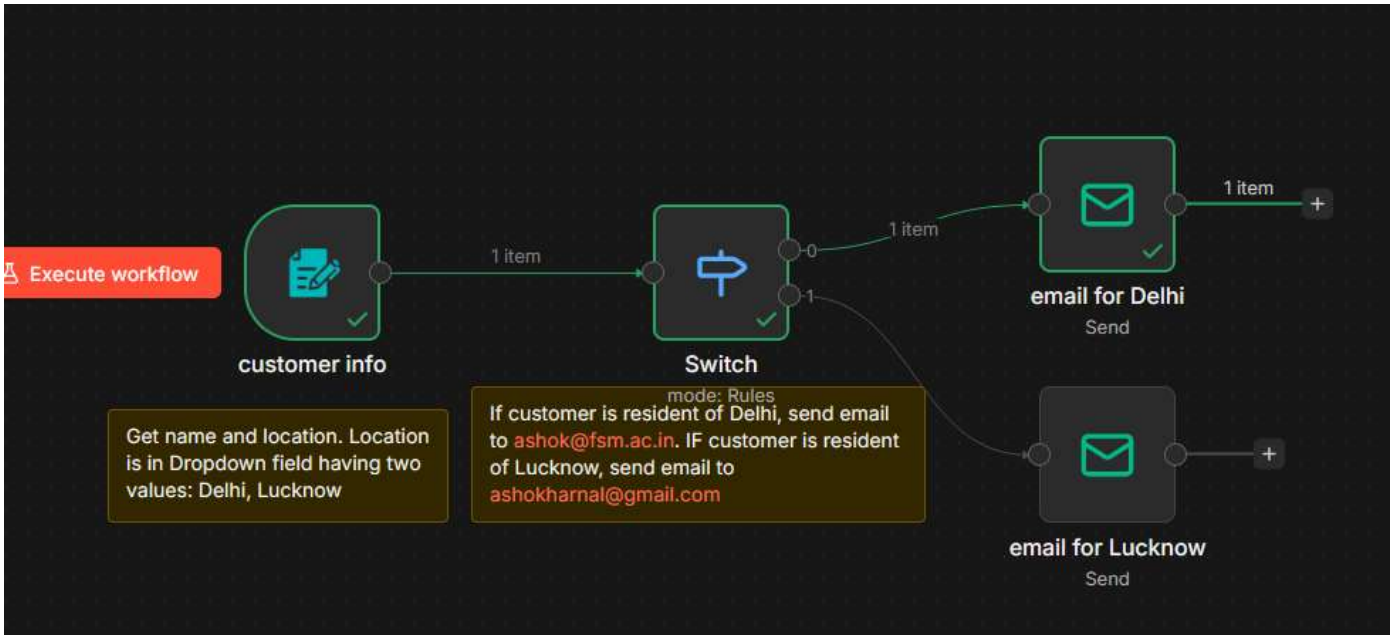


Figure 26: Input element of email is tested in the two rules. Upper rule tests for ashok@fsm.ac.in and the lower rule tests for ccma@fsm.ac.in. In both cases, ***rename*** *your branches appropriately.*


Execute workflow
customer info
1 item
Switch
mode: Rules
0
1
1 item
email for Delhi
Send
1 item
email for Lucknow
Send
Get name and location. Location is in Dropdown field having two values: Delhi, Lucknow
If customer is resident of Delhi, send email to ashok@fsm.ac.in. IF customer is resident of Lucknow, send email to ashokharnal@gmail.com

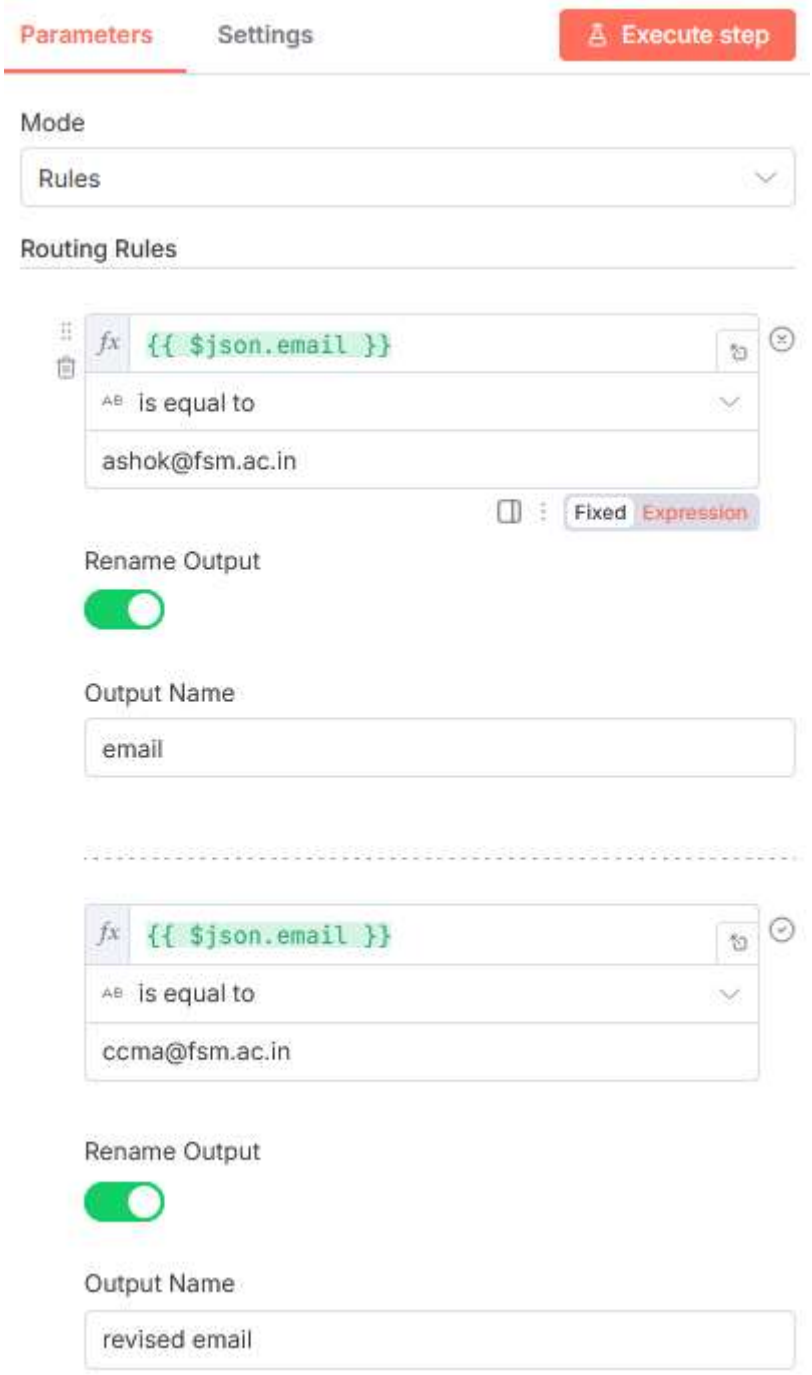


Figure 27: Switch node details

Another sample rule:

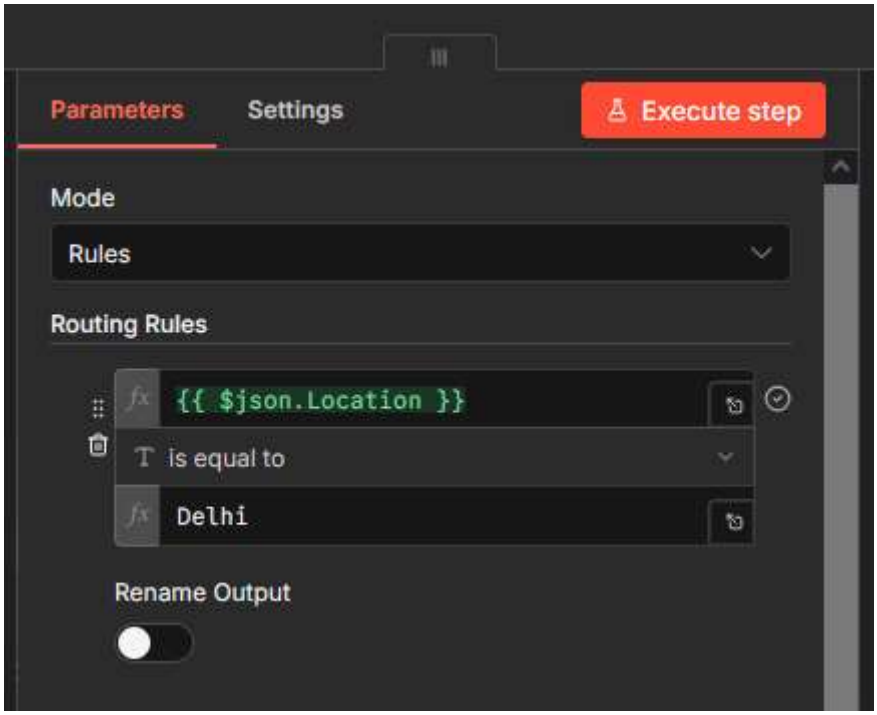


Figure 28: Note Delhi is NOT in double quotes

N. N8n memory error

n8n keeps data in memory while the workflows are running. Creating sub-workflows is a good idea as after a sub-workflow is executed, its memory is released. At times n8n breaks and gives memory error. Error message is about JavaScript heap memory being exceeded. Memory needs to be increased. See this link and this link.

One can assign more memory by changing the environment variable `--max-old-space-size`. This can be done while starting docker, as:

```
docker run -d --name n8n -p 5678:5678 -e NODE_OPTIONS="--max-old-space-size=8000"
docker.n8n.io/n8nio/n8n
```

And, if n8n is directly installed, run n8n as:

```
NODE_OPTIONS="--max-old-space-size=8000" npx n8n
```

To check memory usage, issue top command. Specifically for user ashok, issue top -u ashok :

```
top - 09:33:57 up 21 min,  1 user,  load average: 0.11, 0.08, 0.02
Tasks:  52 total,   1 running,  51 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.5 us,  0.1 sy,  0.0 ni, 99.4 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
   1486 ashok     20   0   38.6g   6.9g  56640 S  14.0  22.9   1:46.81 node
   1605 ashok     20   0    7800   3680   3040 S   0.3   0.0   0:00.31 top
      1 root      20   0  167152  11356   7836 S   0.0   0.0   0:00.81 systemd
      2 root      20   0    3860   1760   1760 S   0.0   0.0   0:00.00 init-systemd(Ub
      6 root      20   0    3876   1820   1760 S   0.0   0.0   0:00.00 init
     61 root      19  -1   47804  14080  13280 S   0.0   0.0   0:00.11 systemd-journal
     90 root      20   0   23020   5888   4608 S   0.0   0.0   0:00.10 systemd-udevd
    127 systemd+  20   0   26280  14240   9120 S   0.0   0.0   0:00.05 systemd-resolve
    128 systemd+  20   0   89364   7040   6240 S   0.0   0.0   0:00.05 systemd-timesyn
    209 root      20   0    4308   2560   2400 S   0.0   0.0   0:00.00 cron
    211 message+  20   0    8588   4000   3680 S   0.0   0.0   0:00.13 dbus-daemon
    223 root      20   0   38088  18400   9920 S   0.0   0.1   0:00.08 networkd-dispat
    226 syslog    20   0  222404   4800   4160 S   0.0   0.0   0:00.02 rsyslogd
    232 root      20   0   16336   6880   6240 S   0.0   0.0   0:00.08 systemd-logind
    253 root      20   0    3240   2080   2080 S   0.0   0.0   0:00.00 agetty
    257 root      20   0    3196   1920   1920 S   0.0   0.0   0:00.00 agetty
    258 root      20   0   15436   9120   7520 S   0.0   0.0   0:00.00 sshd
    322 root      20   0  107164  20960  13120 S   0.0   0.1   0:00.07 unattended-upgr
    359 postgres  20   0  215888  29600  27040 S   0.0   0.1   0:00.05 postgres
    441 postgres  20   0  215808   7448   4800 S   0.0   0.0   0:00.00 postgres
    442 postgres  20   0  215808   8568   5920 S   0.0   0.0   0:00.04 postgres
    443 postgres  20   0  215808  11448   8800 S   0.0   0.0   0:00.01 postgres
    444 postgres  20   0  216376  10168   7200 S   0.0   0.0   0:00.00 postgres
```

Figure 29: `top` command output. RES shows total memory usage by user 'ashok' and the process is `node` (ie nodejs).

Figure 30: Output of command: `top -u ashok`. You can also see `total memory` as also `free memory`

O. CRUD operations on PostgreSQL

Steps:

a. Reconfigure PostgreSQL security:

- Configure */etc/postgresql/16/main/postgresql.conf*. Here '/16/' is the version number. Check the line:

  ```
  cat /etc/postgresql/16/main/postgresql.conf  | grep listen
  ```

  You will get:

  ```
  #listen_addresses = 'localhost'
  ```

- Modify the *listen_addresses* parameter to allow connections from external interfaces. To allow connections from all interfaces, set it to '*'.

  listen_addresses = '*'

- Configure */etc/postgresql/16/main/*pg_hba.conf. For broader access (e.g., from a specific IP range), you can use a CIDR notation. In google you can raise a query: '*how to write 172.30.109.200 with netmask of 255.255.240.0 in cidr notation*'. Or see this link for calculation:

  host all all 172.30.96.0/20 md5

- Restart postgresql

  sudo systemctl restart postgresql

- Create a user, say kumar, and assign him necessary privileges:
  (See this file (at line no 170-211)for all the quick code):

```
sudo useradd -m kumar
sudo passwd kumar
sudo -u postgres psql -c 'create database kumar;'
sudo -u postgres psql -c 'create user kumar;'
sudo -u postgres psql -c 'grant all privileges on database kumar
to kumar;' -d kumar
sudo -u postgres psql -c "alter user kumar with encrypted
password 'kumar';"
sudo -u postgres psql -c " GRANT ALL ON SCHEMA public TO kumar;"
-d kumar
sudo -u postgres psql -c " CREATE EXTENSION vector;" -d kumar

# Add a table and a record
```

```
sudo -u kumar psql -c "CREATE TABLE acars (  brand VARCHAR(255),
model VARCHAR(255),  year INT);"  -d kumar
sudo -u kumar psql -c "INSERT INTO acars (brand, model, year)
VALUES ('Ford', 'Mustang', 1964);" -d kumar
```

- OR as: Create a table in database *kumar*, as:

```
$sudo su kumar
$psql kumar
kumar=> \c kumar
You are now connected to database "kumar" as user "kumar".
kumar=> CREATE TABLE acars (  brand VARCHAR(255), model VARCHAR(255),year INT);
kumar=> INSERT INTO acars (brand, model, year) VALUES ('Dzire','Maruti', 1994);
kumar=> INSERT INTO acars (brand, model, year) VALUES ('Swift','Maruti', 1984);
kumar=> select * from acars ;
```

postgresql quick steps:

```
./psql.sh
create user kavita with password 'kavita' ;
create database kavita with owner kavita ;
\l
\q
sudo useradd -m kavita
sudo su Kavita
psql
\c Kavita
create table goat (id int primary key, name varchar(3000)) ;
\q
./reset_n8n.sh
FORM (with two fields: id and name)==>Postgresql (Insert)
```

b. In n8n create the following very simple network:

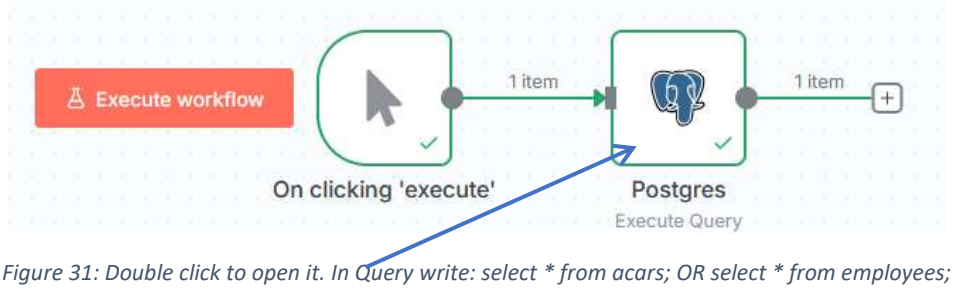


*Figure 31: Double click to open it. In Query write: select * from acars; OR select * from employees;*

Create PostgreSQL credentials, as:

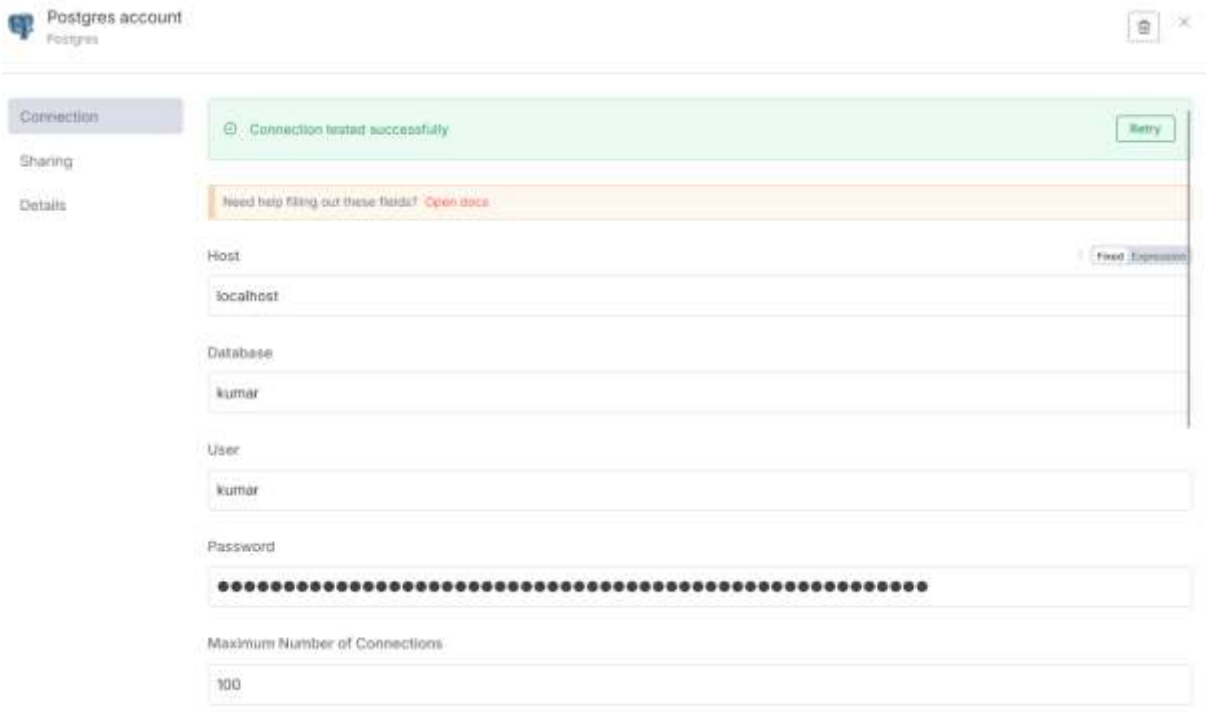


Figure 32: User: kumar; password: kumar; database: kumar

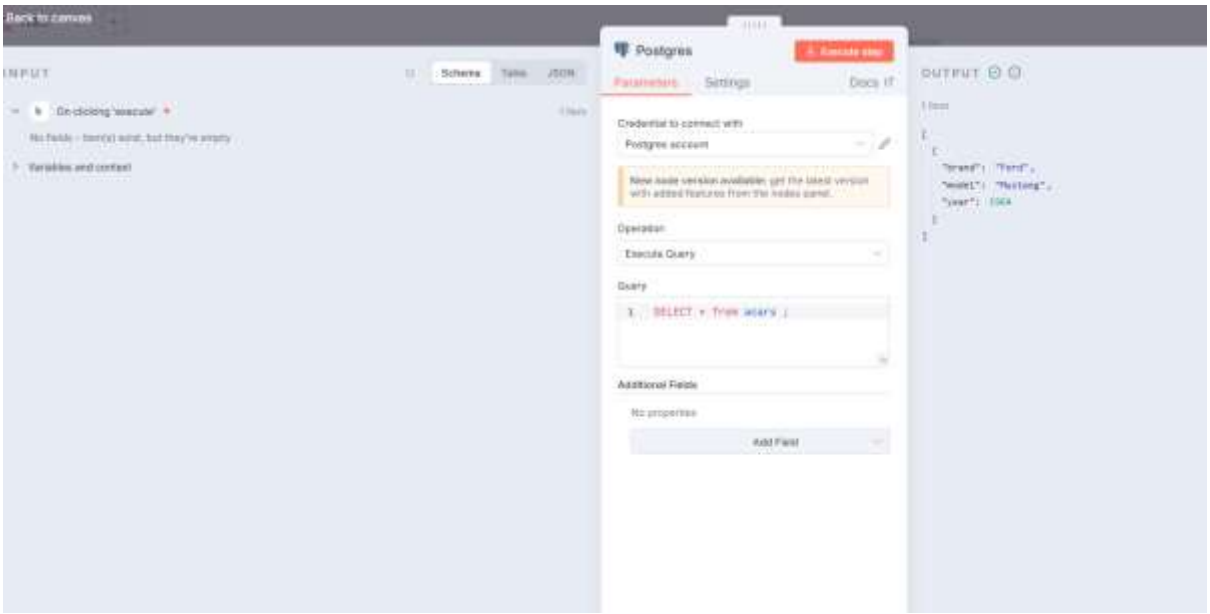


*Figure 33: Query: select * from acars; The result is on the right.*

Extend the above workflow further, step-by-step. Available PostgreSQL nodes are:

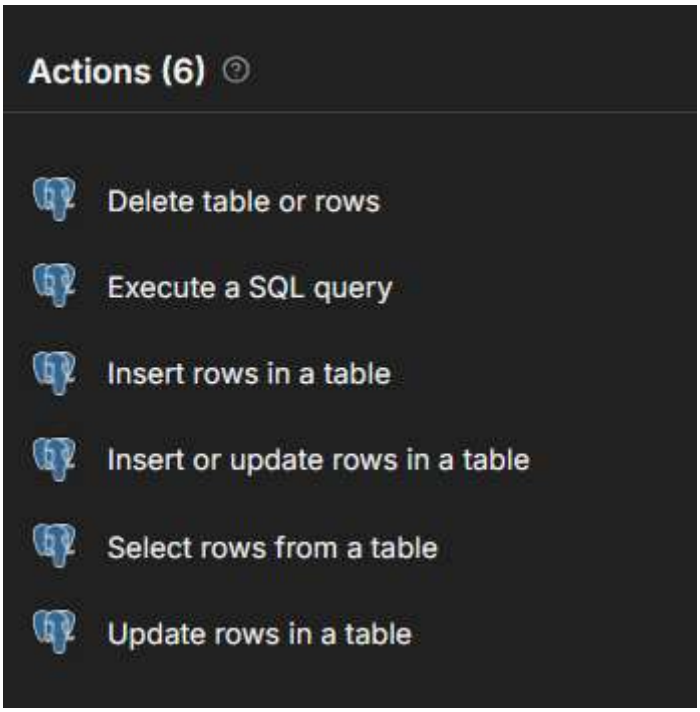


Figure 34: Available PostgreSQL nodes in n8n

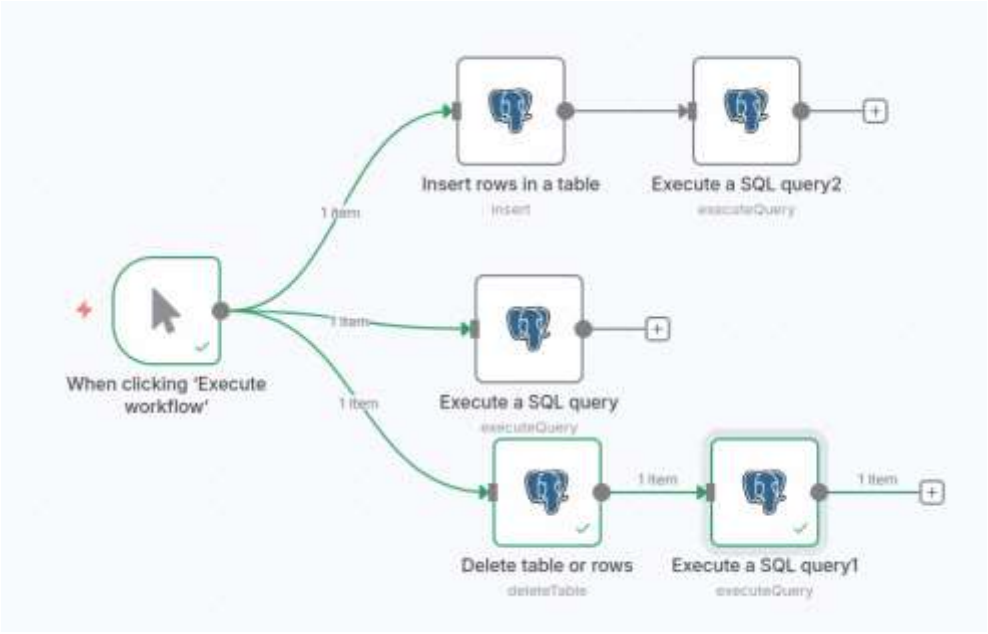


Figure 35; Workflow using different postgres nodes: Execute SQL Query, Insert rows, Delete table of rows

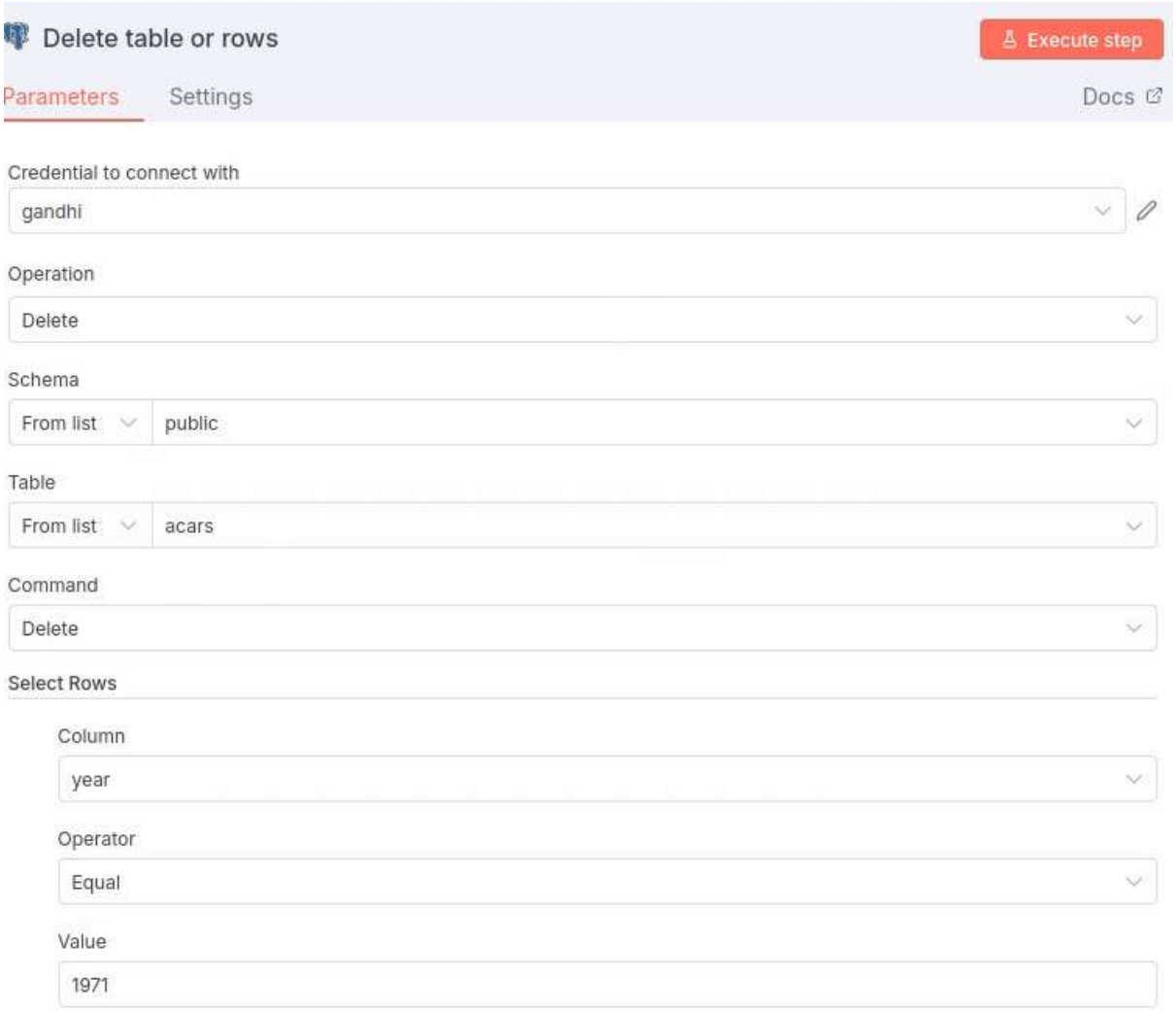


Figure 36: Delete table rows settings

P. Postgres Data insertion with Form:

Create a table in 'ravi' database.

```
sudo sur avi
psql ravi
\c ravi
Create table ravitest (name varchar(20), email varchar(20)) ;
```

Now use form to Insert this data into postgres table. And after inserting, execute query:

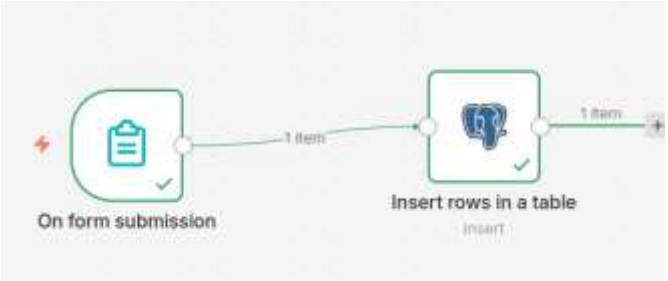


Figure 37: Form entering data into postgres

```
select * from ravitest.
```

```
ravi=> create table ravitest  (name varchar(20), email varchar(20)) ;
CREATE TABLE
ravi=> select * from ravitest;
 name  |      email
-------+-----------------
 ashok | ashok@fsm.ac.in
(1 row)
```

Figure 38: Data inserted from form into postgres

Q. SQL generation -- AI agent and postgres

Simple SQL agent

Database can be postgres, or SQLite or MySQL. In the terminal use `./psql.sh`, then connect to ravi (`\c ravi`). Then issue commands: `\l and \d` or `select * from spj;` *Execute Query*: {{ $fromAI('sql_statement') }}. Model: qwen3.5:0.8b also works.

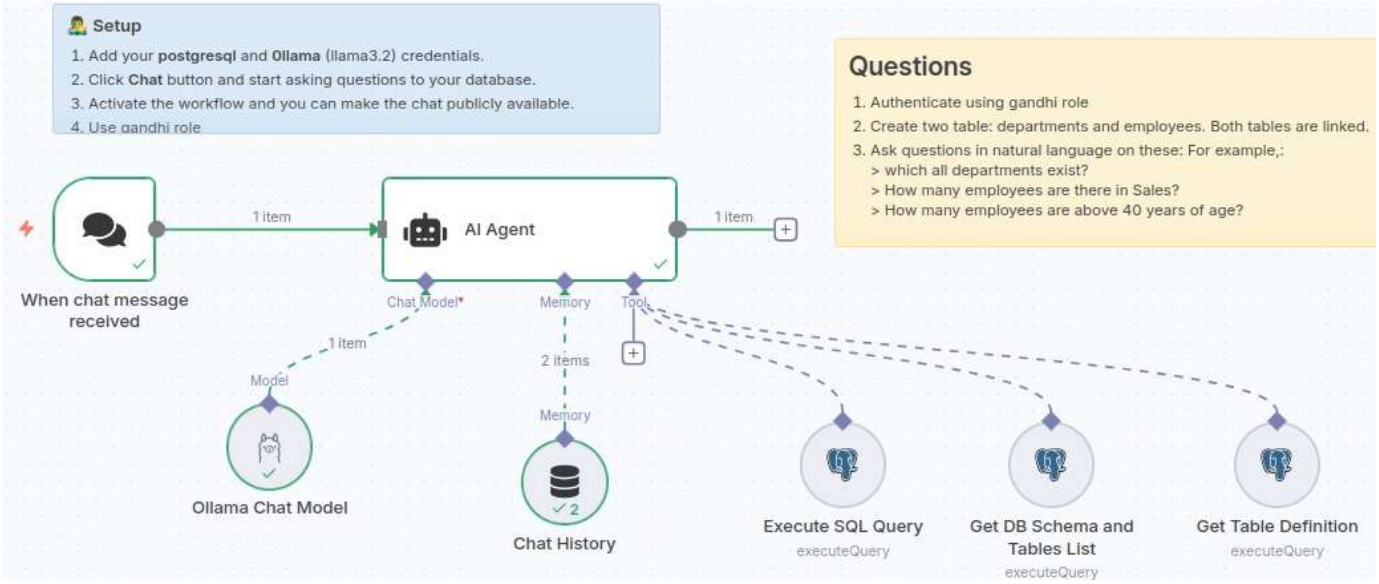
Query1: Count the number of rows in table spj ; **Query2**: List the names of tables in database ravi

Figure 39: User is ravi*, password is ravi and database is ravi. The database contains the standard C J Date's tables: s,p,j,spj and an additional table : distributors. Model: qwen3.5:0.8b also works*

Complex SQL ai agent (with postgres)

https://n8n.io/workflows/1954-ai-agent-chat/

See SQL code here.



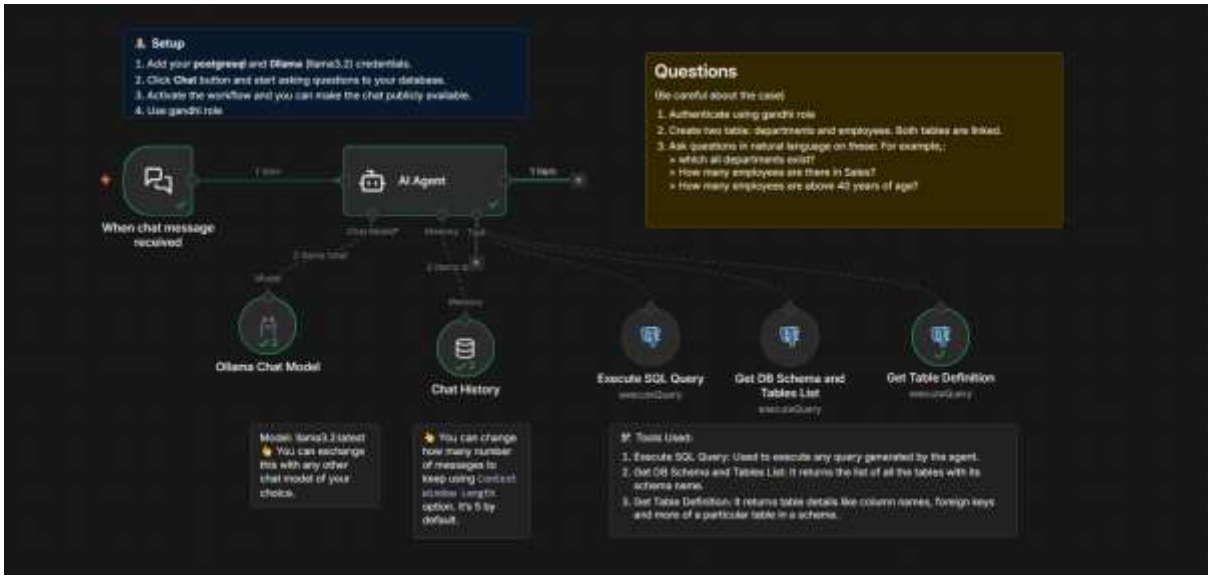


Figure 40: Another view with theme change

Replace chat history by PG Chat memory. In PG Chat memory, you can specify your own table name.

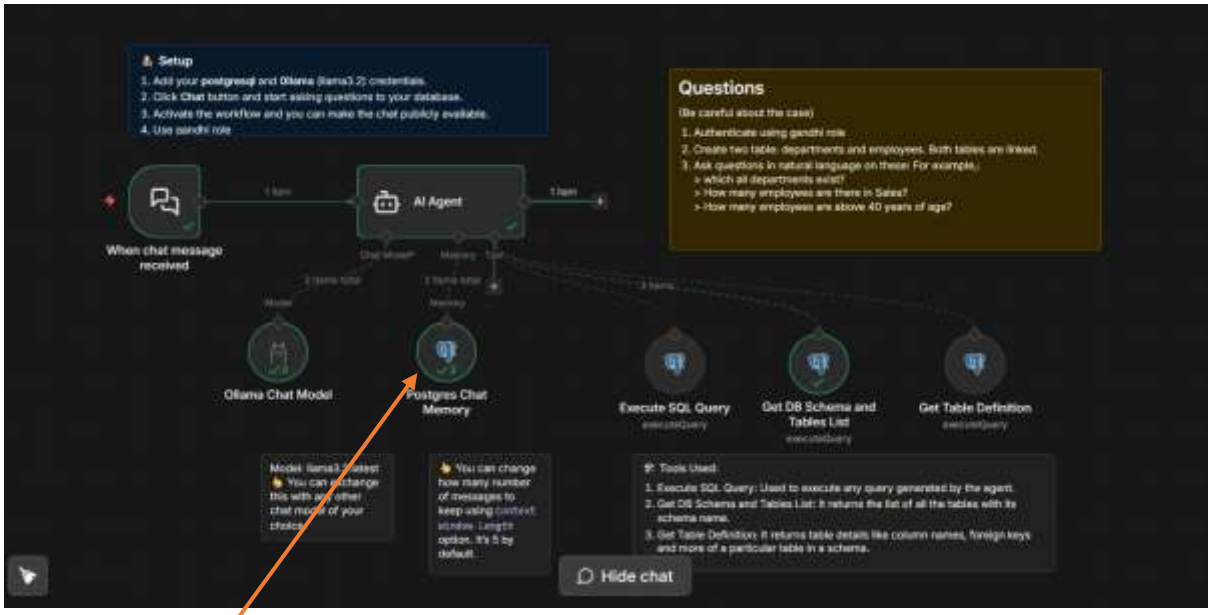


Figure 41: Replace chat history by PG Chat memory. The PostgreSQL account or credentials can be the same as in other PG nodes. In PG Chat memory, you can specify your own table name.

System prompt of AI agent:

> You are DB assistant. You need to run queries in DB aligned with user requests.
> Run custom SQL query to aggregate data and response to user. Make sure every table has schema prefix to it in sql query which you can get from `Get DB Schema and Tables List` tool.
> Fetch all data to analyse it for response if needed.

Tools

- Execute SQL query- Executes any sql query generated by AI
- Get DB Schema and Tables List - Lists all the tables in database with its schema name
- Get Table Definition - Gets the table definition from db using table name and schema name"

Execute SQL Query node:

a. Tool Description→Set Manually
b. Description:
   Get all the data from Postgres, make sure you append the tables with correct schema. Every table is associated with some schema in the database.
c. Operation→Execute Query

```
{{ $fromAI("sql_query", "SQL Query") }}
```

Meaning of this expression is as below:

AI Overview हिन्दी

In n8n, the expression `{{ $fromAI("sql_query", "SQL Query") }}` is **a special, dynamic function used within AI-powered workflows to extract a specific piece of information generated by an AI Agent (usually an LLM like GPT-4) and use it within a subsequent node**.

Here is a breakdown of what this specifically means:

- `$fromAI(...)` : This is a method designed for AI agent tools to retrieve data generated by the LLM that is not part of the final text output, but rather a structured output intended for tool execution.
- `"sql_query"` : This is the **key** or variable name that the AI Agent was instructed to generate. It tells the expression which part of the AI's output to look for.
- `"SQL Query"` : This is a human-readable description for the variable, often used for debugging or documentation within the node parameters. CloudQuery +3

Context of Use: Database Agents

This expression is almost always used in the **Execute SQL Query** node (a PostgreSQL or MySQL tool) within an AI Agent workflow.

1. **User Input:** The user asks a question in natural language, e.g., *"How many users signed up today?"*.
2. **AI Agent:** The LLM interprets the question and generates a JSON object containing the SQL query, e.g., `{ "sql_query": "SELECT count(*) FROM users WHERE created_at = '...';" }`.
3. `$fromAI` **Function:** The SQL node uses `{{ $fromAI("sql_query", "SQL Query") }}` to extract just the SQL string (`SELECT ...`) from that JSON object and execute it against the database.

Get DB Schema node

a. Tool Description→Set Manually
b. Description:
   Get list of all tables with their schema in the database.
c. Operation→Execute Query

```
SELECT
```

```
    table_schema,
    table_name
FROM information_schema.tables
WHERE table_type = 'BASE TABLE'
    AND table_schema NOT IN ('pg_catalog',
'information_schema')
ORDER BY table_schema, table_name;
```

Get Table Definition

d. Tool Description→Set Manually
e. Description:
   Get table definition to find all columns and types
f. Operation→Execute Query

```
select
  c.column_name,
  c.data_type,
  c.is_nullable,
  c.column_default,
  tc.constraint_type,
  ccu.table_name AS referenced_table,
  ccu.column_name AS referenced_column
from
  information_schema.columns c
LEFT join
  information_schema.key_column_usage kcu
  ON c.table_name = kcu.table_name
  AND c.column_name = kcu.column_name
LEFT join
  information_schema.table_constraints tc
  ON kcu.constraint_name = tc.constraint_name
  AND tc.constraint_type = 'FOREIGN KEY'
LEFT join
  information_schema.constraint_column_usage ccu
  ON tc.constraint_name = ccu.constraint_name
where
  c.table_name = '{{ $fromAI("table_name") }}'
  AND c.table_schema = '{{ $fromAI("schema_name") }}'
order by
  c.ordinal_position
```

Create tables

(See this file (at line no 170-211)for all the quick code):

```
CREATE TABLE departments (
                        department_id int PRIMARY KEY,
                        department_name VARCHAR(100) NOT NULL
                        );

CREATE TABLE employees (
                    employee_id int PRIMARY KEY,
                    employee_name VARCHAR(100) NOT NULL,
                    age float,
                    departmentId INT REFERENCES departments (department_id)
                    );
```

INSERT Data:

```
INSERT INTO departments (department_id,department_name) values
(1, 'Human Resources'),
(10,'Finance'),
(20,'IT'),
(30, 'Operations') ;

INSERT INTO employees (employee_id,employee_name, age, departmentId) values
(1,'Alice', 30.4, 10),
(2,'Bob',50.8, 10),
(3,'sudha', 39.0, 20),
(4,'Charlie',34.0, 30);
```

R. Data transformation

S. Sub-workflows in n8n

T. Using webhook in n8n

U. Slack

How to get slack api or token from slack. Here are the steps:

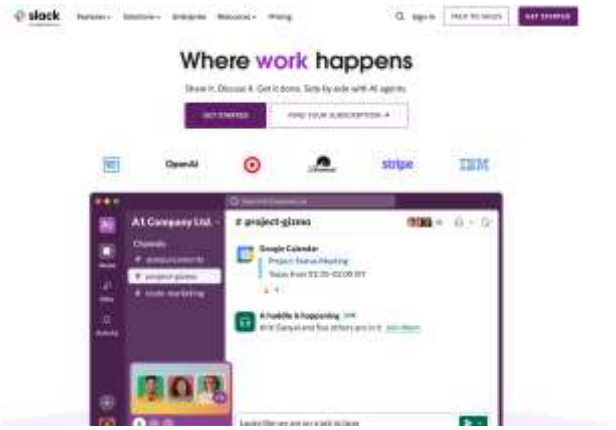


Figure 42: Sign in using your email.

Figure 43: Create a new Workspace of your own where your team will work

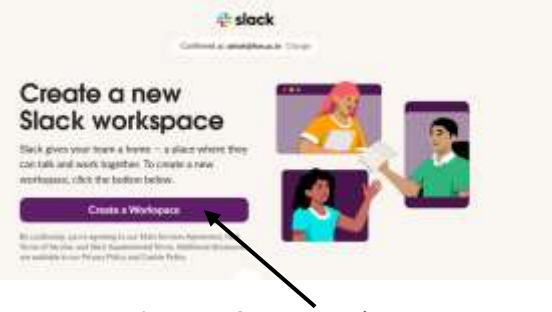


Figure 44: Create a Workspace

Figure 45: Fill in some details about your workspace. Name your team and click Next

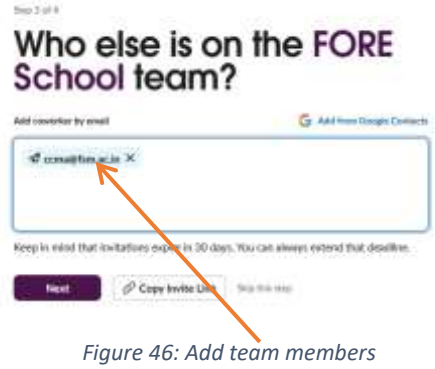


Figure 46: Add team members

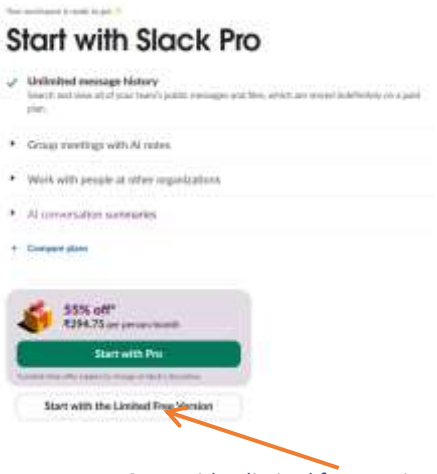


Figure 47: No. Start with a limited free version

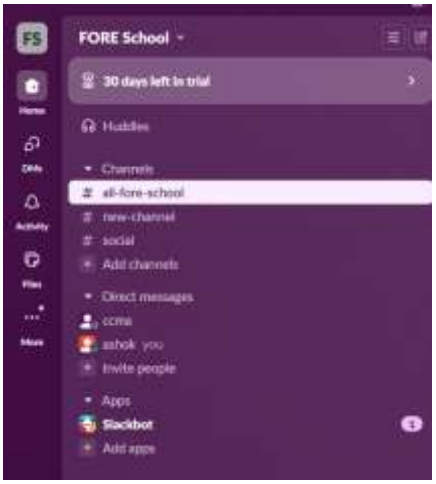


Figure 48: Three channels, one app

What are Slack channels:

A Slack channel is a dedicated space within a Slack workspace where teams can collaborate, communicate, and share information related to a specific topic, project, or team. Channels can be public, allowing anyone in the workspace to join, or private, requiring explicit invitation. Key features of channels are:

Collaboration: Channels facilitate communication, file sharing, and even video/audio calls.

Searchability: Messages and files within a channel are searchable, making it easy to find past conversations and information.

Customization: Channels can be created for various purposes, such as project updates, team discussions, or specific topics.

A company might have a public channel for general announcements (#general), a private channel for sensitive HR matters (#hr-private), and a project-specific channel for a new product launch (#product-launch).

What is an app

Apps connect other software that you use (such as Google Calendar, OneDrive or one of your company's internal tools) to Slack. With all your tools in one place, you can streamline work and help people in your workspace collaborate more effectively.

What you need to know

- There are a few different types of apps that you may see in Slack – built by Slack, third parties or your own team. How an app was built determines how it can be installed and managed in a workspace, as well as where and how you'll be able to interact with it.
- By default, any workspace member can install apps, but owners and admins can choose to restrict this permission. Once an app is installed to a workspace, any member can connect their account to use it.
- Before installing an app from the Slack Marketplace, you can review its privacy policy and security and compliance information (if submitted by the app's developer) from the app page. We recommend only choosing services that you trust when installing apps to Slack.

V. Pinecone vector store:

1. Create a free account (say, using google)
2. Create an API key
3. Create a (vector) index

Understanding Pinecone indexes:

In Pinecone, an index is the primary organizational unit for storing and querying vector data. Think of it as a table in a database, but specifically designed for efficient similarity searches on vectors. Indexes can accept, store, and serve queries on vectors, as well as perform other vector operations. Here's a more detailed breakdown:

Storage and Organization:

Indexes hold the vector embeddings of your data, allowing you to store and manage them in a structured way.

Key Parameters dimension etc:

When creating an index, you'll need to define its name, the dimensionality of the vectors it will store, and the similarity metric (e.g., cosine, Euclidean).

Types of Indexes for scaling out:

Pinecone offers serverless and pod-based indexes, with different options for scaling and performance.

Namespaces:

Within an index, you can further organize data using namespaces, allowing you to isolate queries to specific subsets of your data.

Similarity Search:

Pinecone indexes are built for efficient similarity searches, meaning you can quickly find vectors that are similar to a given query vector.

Highest Level:

It's the top-level container for your vector data within Pinecone, similar to a table in a relational database.

Creation and Management:

Indexes can be created via the Pinecone UI or programmatically using their API.

Metadata:

Pinecone indexes can store associated metadata with each vector, enabling filtering and more complex search conditions.

Metadata in Pinecone

Every record in an index must contain an ID and a vector. In addition, you can include metadata key-value pairs to store additional information or context. When you query the index, you can then include a metadata filter to limit the search to records matching a filter expression. Searches without metadata filters do not consider metadata and search the entire namespace.

See LlamaIndex example for Chromadb as to how metadata filter works.

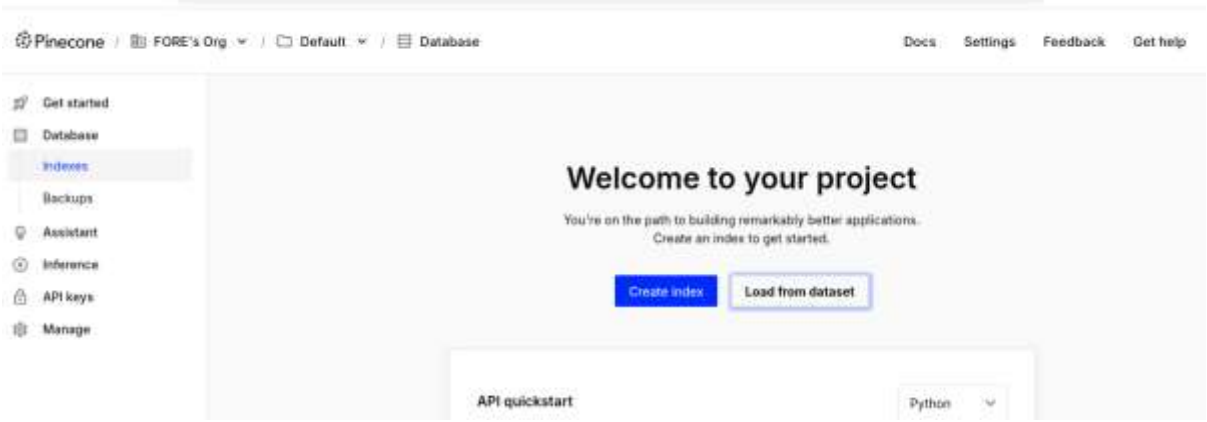


Figure 49: Click Create Index to begin creating an index

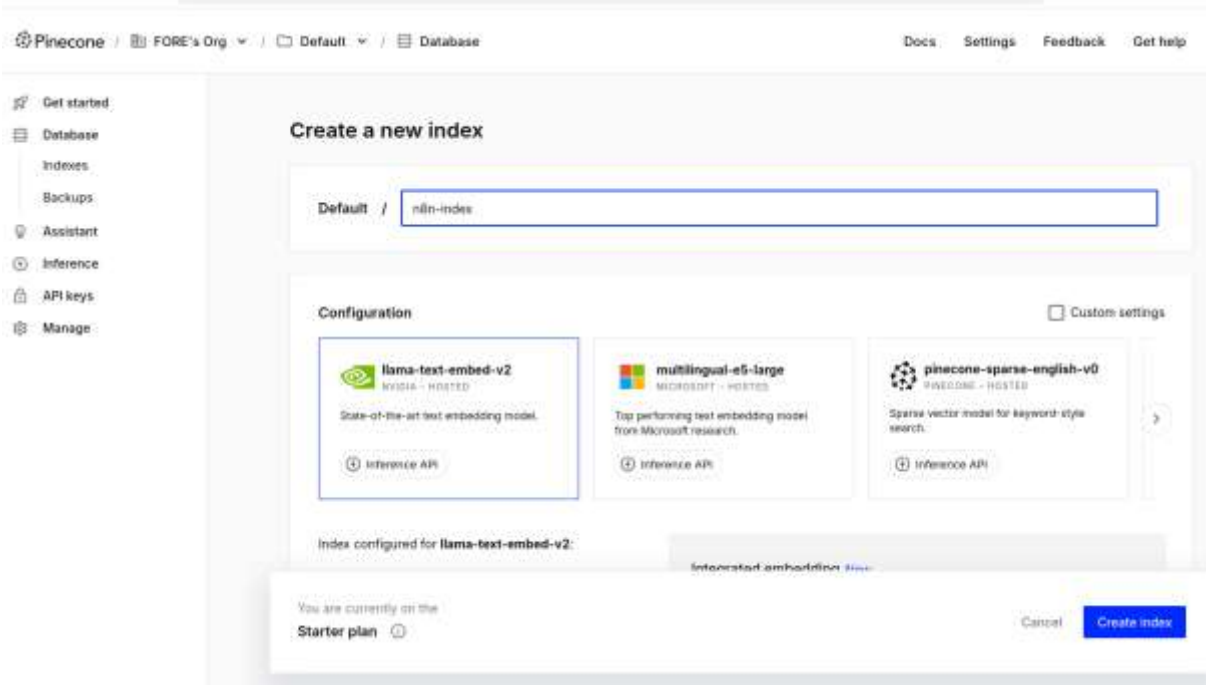


Figure 50: Select an index and select a Configuration for index (such as Vector Dimension: 768, 1024 etc). See figure below

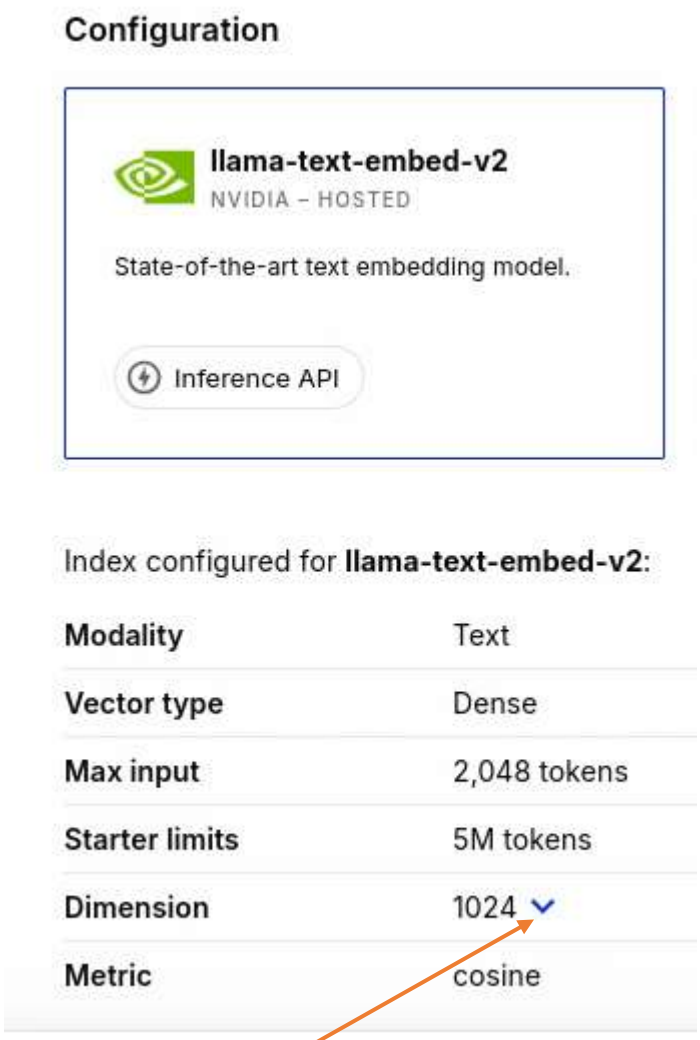


Figure 51: Set vector Dimension as per your embedder (click Down-arrow). For example, nomic-embed-text has a vector dimension of 768 AND NOT of 1024. We have used 768

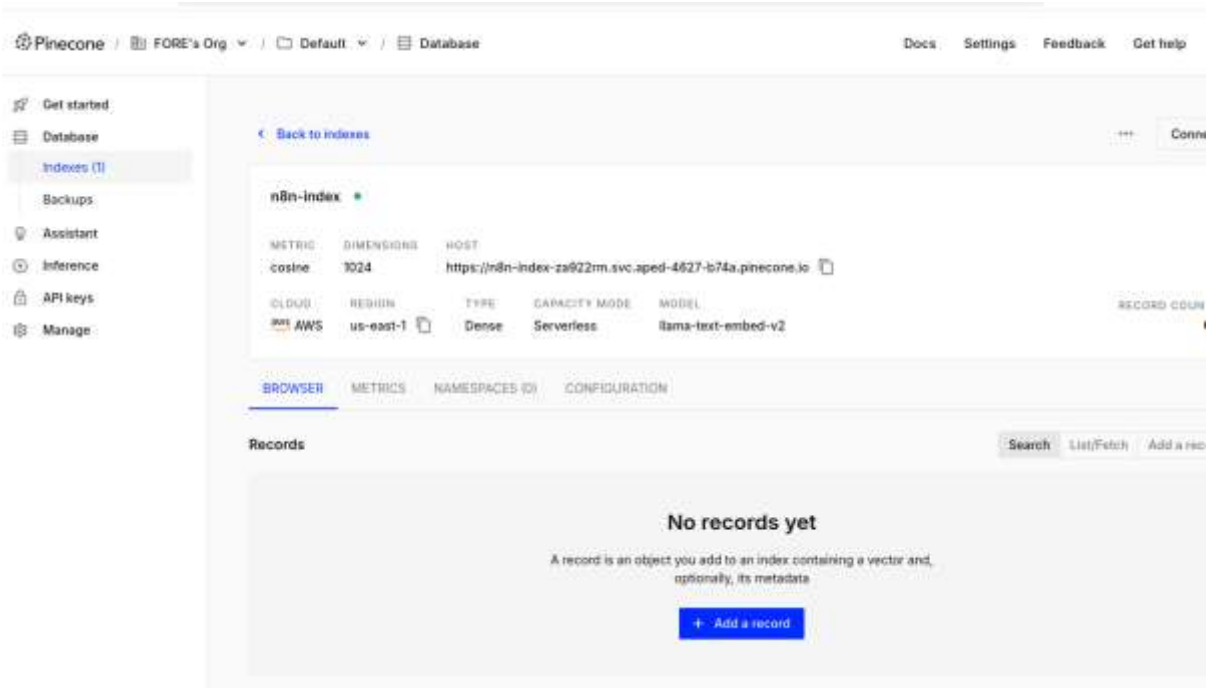


Figure 52: Create index. Records will come from n8n workflow

Note that selected embedder will also decide max input tokens. The character splitter chunk size will affect the amount of **input tokens** you're trying to get. For the embedding model of. *mxbai-embed-large* (vector dimension: 1024), max input token limit is 512.

Our dataset is *students.json* on GitHub, [at site]

Figure 53: Extract from file students.json

Here are two rows from the data:


```
{"_id":0,"name":"aimee
Zank","scores":[{"score":1.463179736705023,"type":"exam"},{"score":11.78273309957772,"type":"q
uiz"},{"score":35.8740349954354,"type":"homework"}]}
```

```
{"_id":1,"name":"Aurelia
Menendez","scores":[{"score":60.06045071030959,"type":"exam"},{"score":52.79790691903873,"type
":"quiz"},{"score":71.76133439165544,"type":"homework"}]}
```


W. Simple RAG with n8n

File: 'Simple RAG flow.json'

We use '*n8n Form trigger*' node as file uploader. The vector store is in-memory vector store: Simple Vector Store. File used is: *bertrandRusselEssays*.txt

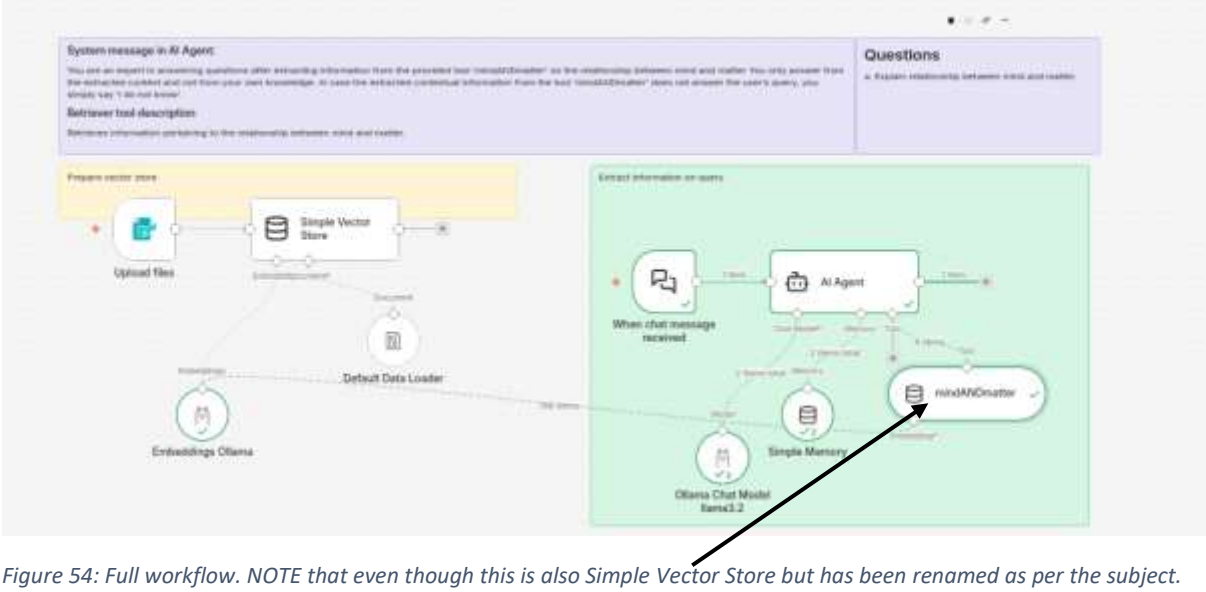


Figure 54: Full workflow. NOTE that even though this is also Simple Vector Store but has been renamed as per the subject.

Messages are as follows:

System Message:

> You are an expert in answering questions after extracting information from the provided tool 'mindANDmatter' on the relationship between mind and matter You only answer from the extracted context and not from your own knowledge. In case the extracted contextual information from the tool 'mindANDmatter' does not answer the user's query, you simply say 'I do not know'.

Tool Des:

> Retrieves information pertaining to the relationship between mind and matter.

Its two sub-workflows are as follows:

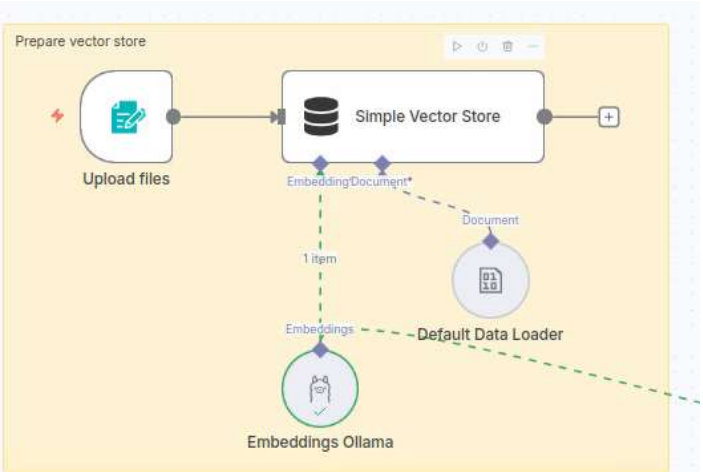


Figure 55: Creating a vector store

Upload files node (n8n form):

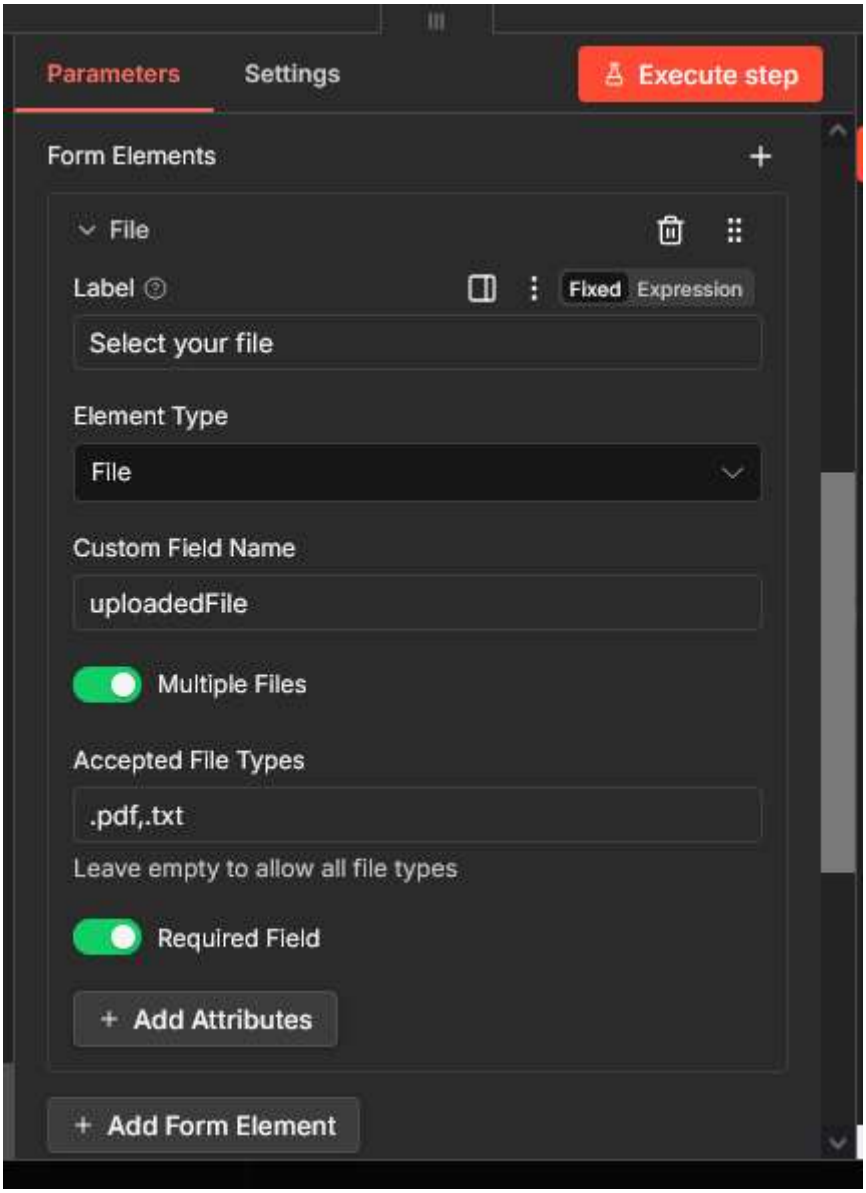


Figure 56: File upload with n8n form. Allow both pdf and txt files upload. Easy to upload Paul Graham essay on Good Wrining.

Simple vector store on the left side of workflow

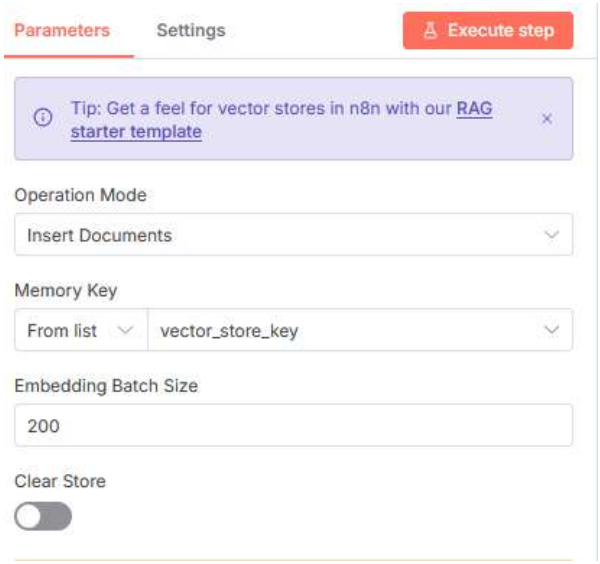


Figure 57: Simple Vector store configuration

Default Data loader

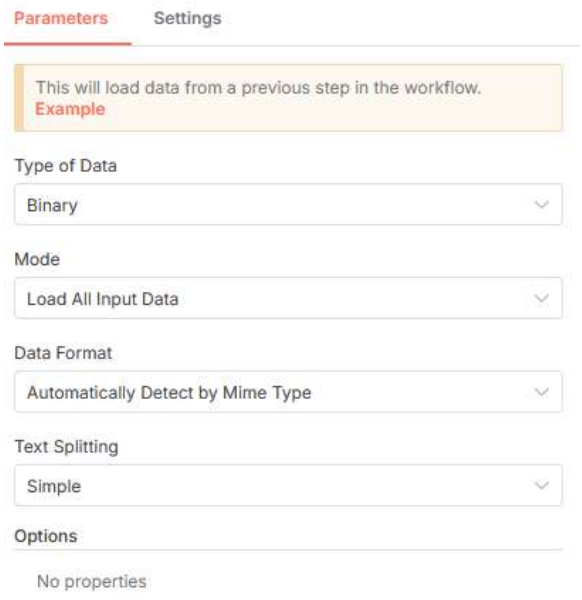


Figure 58: Default data loader. Type of data is Binary (the other choice is JSON).. Text splitting is Simple

AI Agent has no special system prompt. Default is OK. But you should change. See before the suggested system message.

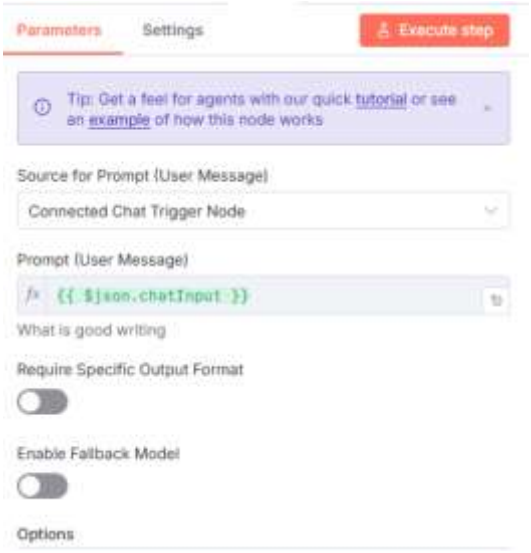


Figure 59: No special System prompt in AI Agent

Simple vector store below AI Agent. Should be named properly.

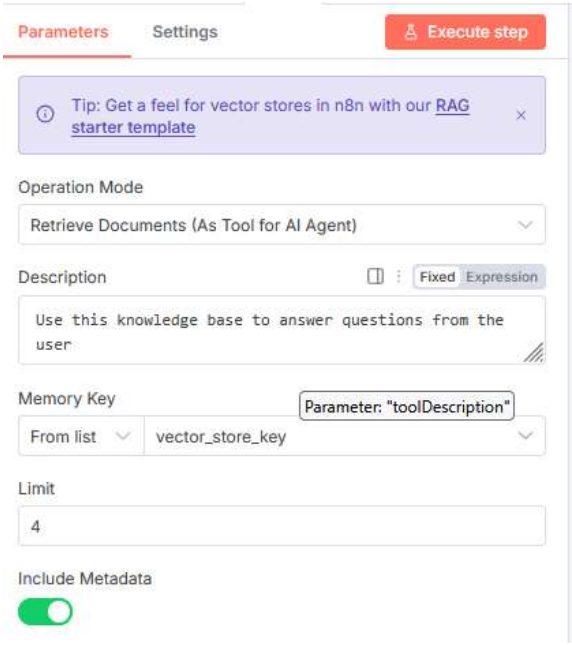


Figure 60: Simple vector store on AI Agent side

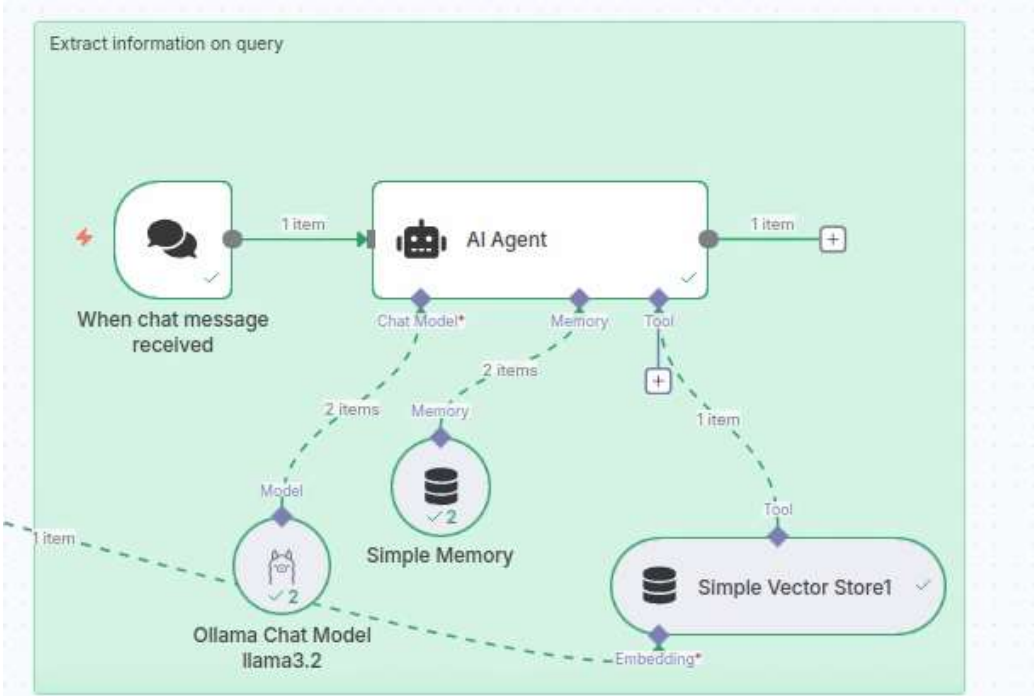


Figure 61: Querying a vector store

System message of AI Agent and vector tool description are as follows:

System message in AI Agent:
You are an expert in answering questions after extracting information from the provided 'Simple Vector Store1' on the game of football. You only answer from the extracted context and not from your own knowledge. In case the extracted contextual information from 'Simple Vector Store1' does not answer the user's query, you simply say 'I do not know'.

Retriever tool description
Retrieves information pertaining to the game of football

Figure 62: System message for AI Agent

X. RAG with Qdrant

This works better than the above.. See [this link](). Quick two lines code to install qdrant(copy and paste):

```
# 1
        mkdir -p ~/qdrant_storage
# 2.
        docker run -d -p 6333:6333 -p 6334:6334 -v ~/qdrant_storage:/qdrant/storage  --name
        my_qdrant qdrant/qdrant
# 3
        Next time start as:
               docker start my_qdrant
```

Then create the following two workflows.

Hints:

a. '*Default Data Loader*' must have '*Binary*' set rather that the default '*Json*'.
b. Give a proper *System Message* in AI Agent. System message mentions the name of tool to be used ('`retriever`', *for example*) and when to be used. (*You are a helpful assistant. When any question is asked get its answer from 'retriever' tool. Under no circumstances answer a question or add to it from your own knowledgebase--only use the 'retriever' tool to answer it.*)
c. Name the Qdrant vector tool also as: '*retriever*'.
d. Use *mxbai-embed-large* OR `bge-m3`(vector dimension: 1024) as embedding model and chunk size as: 200/20
e. Use Ollama cloud models. To use ollama cloud models, first execute '`ollama signin`' ; then go to the web-page and sign into it and `Connect` your device. Next, in the console execute '`ollama run minimax-m3:cloud`'. Then say, `/bye`. From now on, this model will appear in '`ollama list`'.

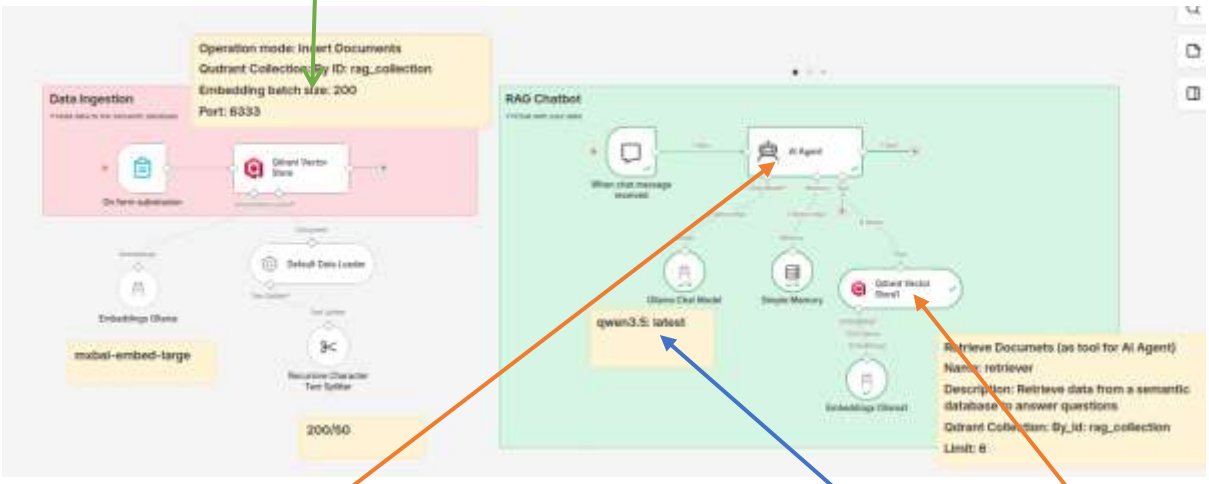


Figure 63: Write a proper System Message. The Collection ID name should be the same in both the branches. Also give a proper name to Retriever tool (such as: 'retriever'). Or, use a small ollama cloud model: minimax-m3:cloud; llama3.2 also works.

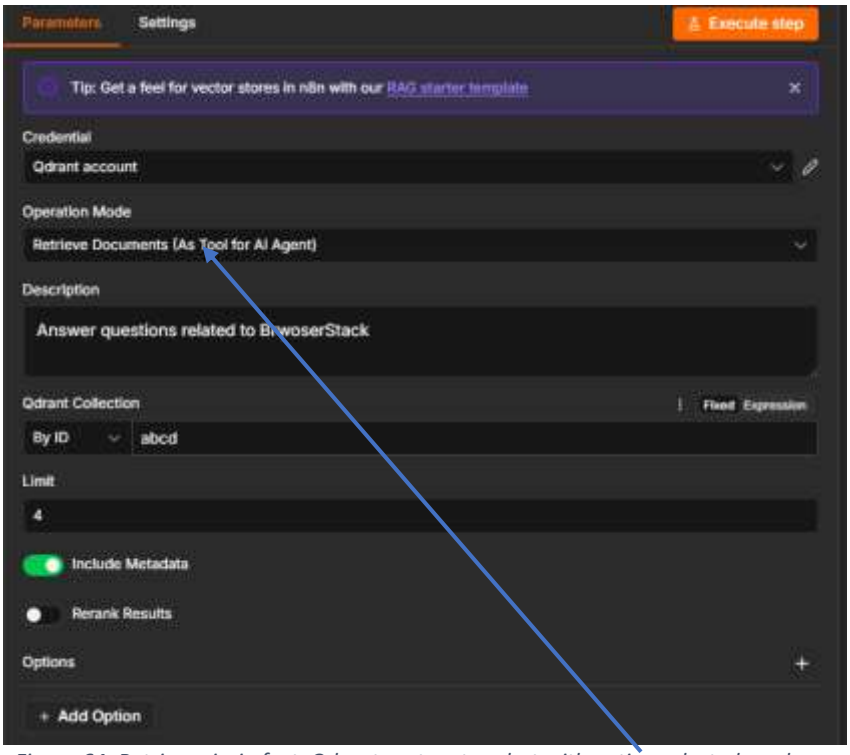


Figure 64: Retriever is, in fact, Qdrant vector store but with option selected as above.

Here use `mxbai-embed-large` embedding model

Key Differences Comparison

Feature	mxbai-embed-large	nomic-embed-text (v2)
Dimensions	1,024	768
Context Window	512 tokens	8,192 tokens
Parameters	~334 Million	~137 Million
Model Size	~670 MB	~274 MB
Best For	High accuracy, RAG with smaller chunks	Fast, local, long document indexing
MTEB (Overall)	Higher (~64)	High (~62)

Figure 65:Use mxbai-embed-large embedding model in the above experiment. Keep chunk size low (200/50).

Y. RAG with Pinecone

N8n model files: n8n/2025/Rag Flow-I.json and Revised rag flow-II.json

Dataset file: *students.json* OR see Try this. For json data, different chunking strategies can be used. See google.

Using Pinecone

Log-into it with google and get API key. Then proceed to create index as follows:

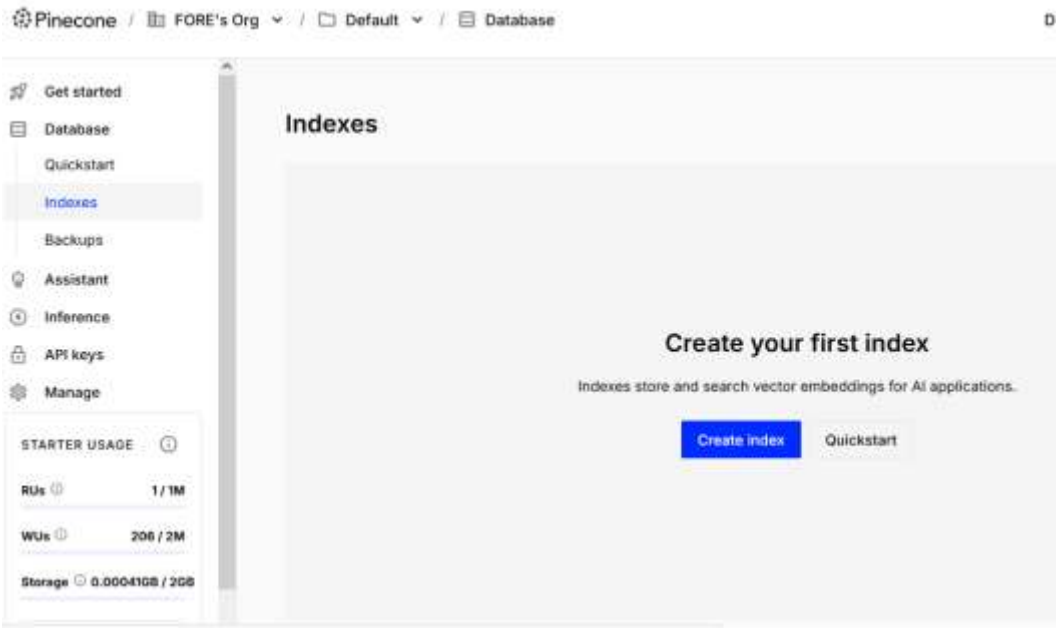


Figure 66: Click Create Index

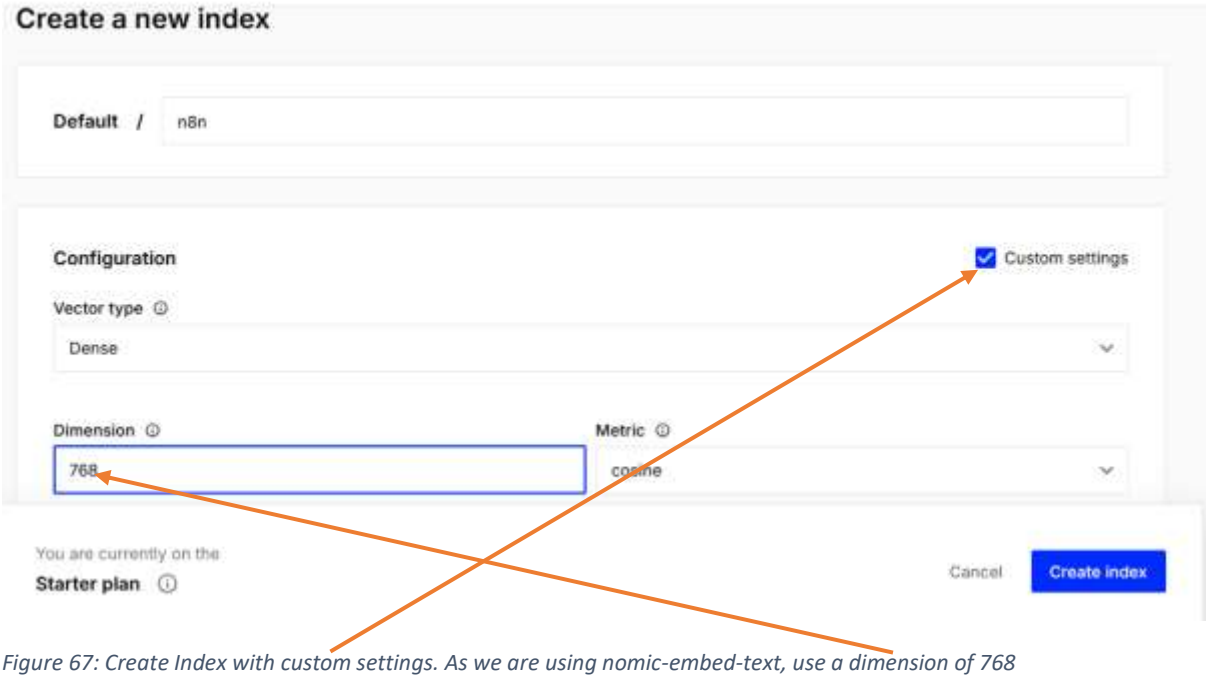


Figure 67: Create Index with custom settings. As we are using nomic-embed-text, use a dimension of 768

See this link for the detailed blog. Vectorization may take a very long time depending upon the input size. As vectorization of files may take a long time, a smaller json file is on GitHub at this link. This file has 35 json records.

Experiment in the class with different students using varying chunk size and maybe different embedder also. Check which embedders are good for json files.

bge-m3: 1024 ➔ Gives Nan error
Nomic-embed-text: 768 ➔ Not that good results
mxbai-embed-large: 1024 ➔ Gives good results (Search for 'Bao Ziglar' scores)

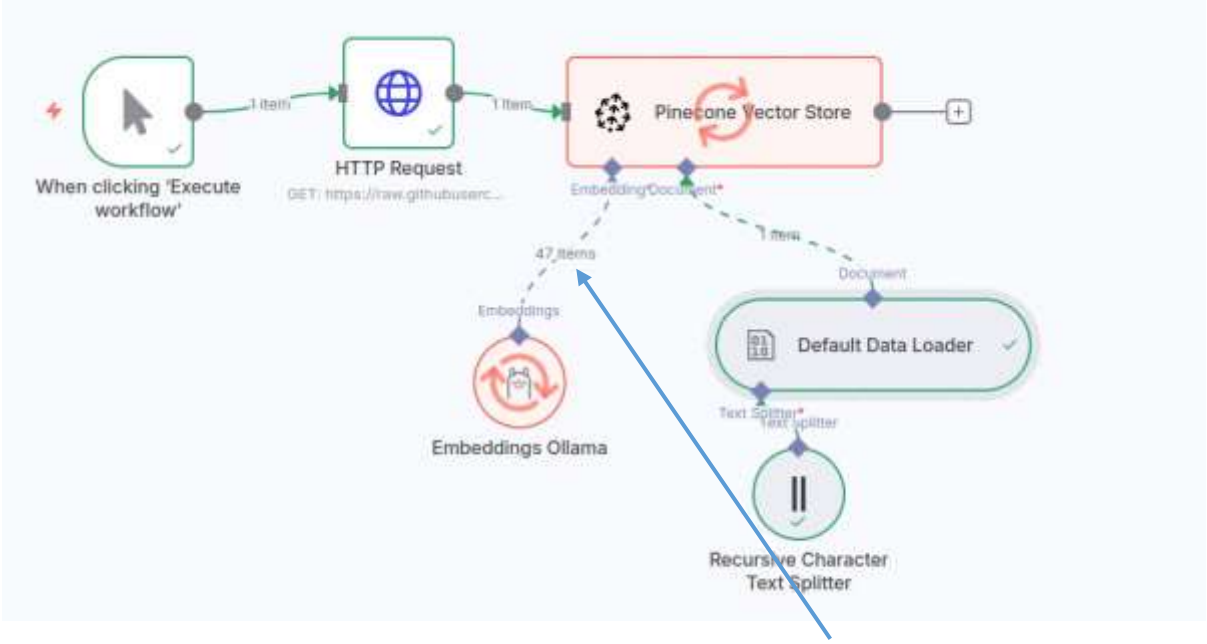


Figure 68: Our workflow to save data to vector store. 47 items possibly mean that 47 chunks were processed till now. Chunk size for Recursive text splitter is 512 as selected embedder is:

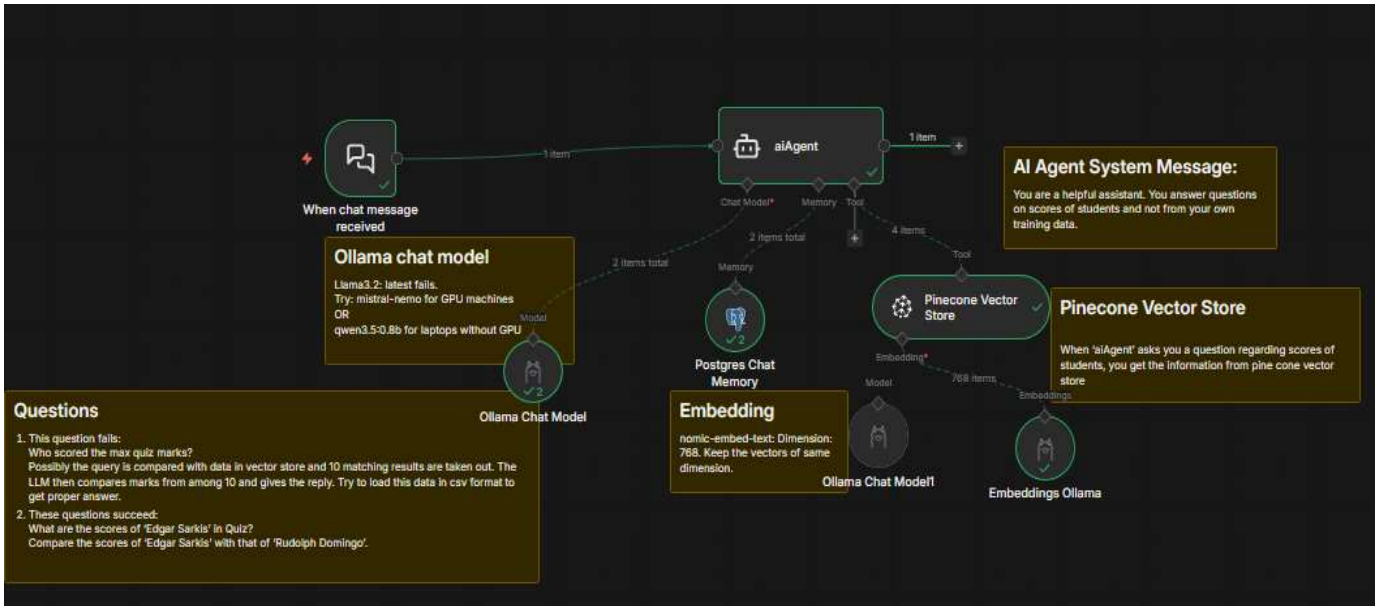


Figure 69: Reading vector data from Pinecone vector store

f. Here are the models used:

Embedding:	*nomic-embed-text* (ollama). Dimensions: 768
Chat ollama:	*mistral-nemo* (ollama)
	qwen3.5:0.8b (for laptops with CPU)

g. Questions asked:

Questions:

- What are the scores of 'Edgar Sarkis' in Quiz?
- Compare the scores of 'Edgar Sarkis' with that of 'Rudolph Domingo'

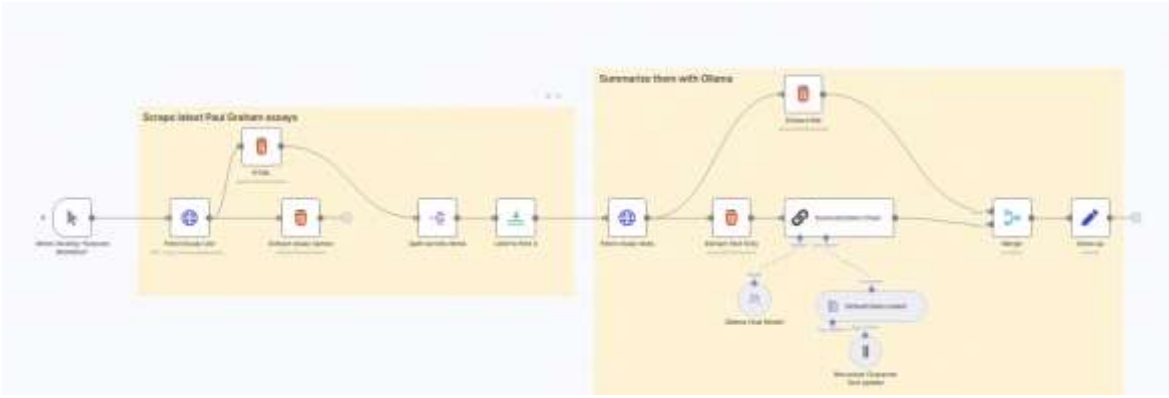
h. AI Agent System message. NOTE NO TOOL NAME MENTIONED HERE??

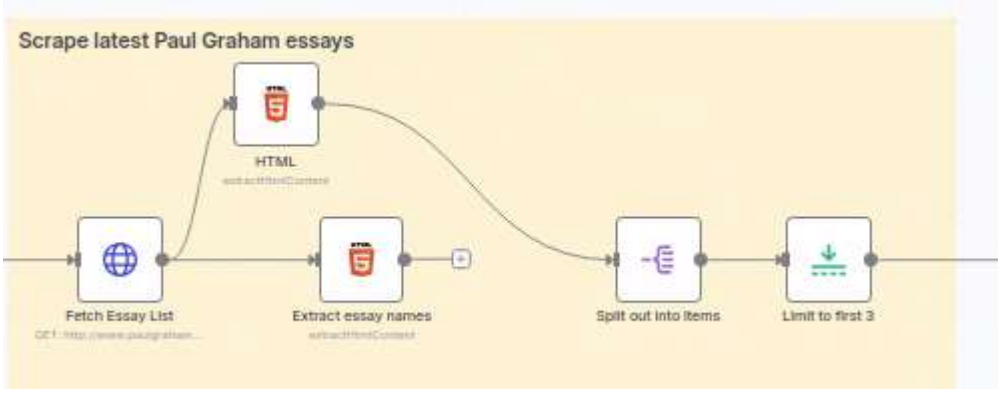
AI Agent System Message:
You are a helpful assistant. You answer questions on scores of students and not from your own training data.

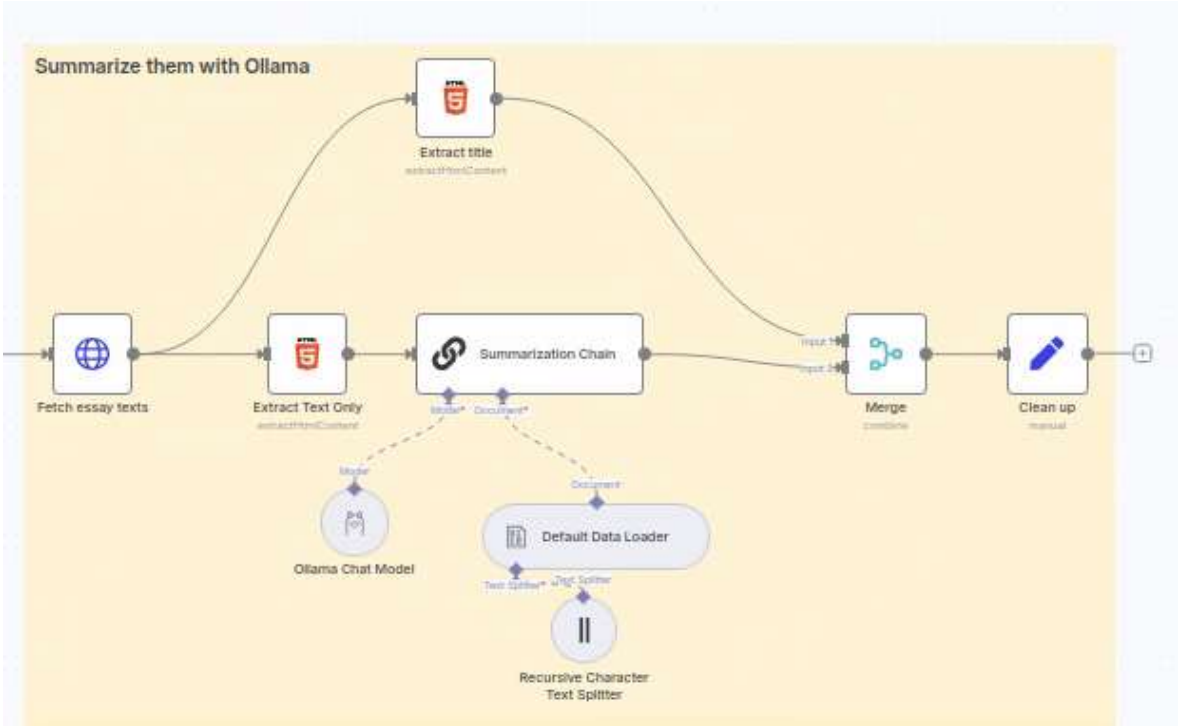
i. Pinecone Vector Store Description. TOOL NAME?

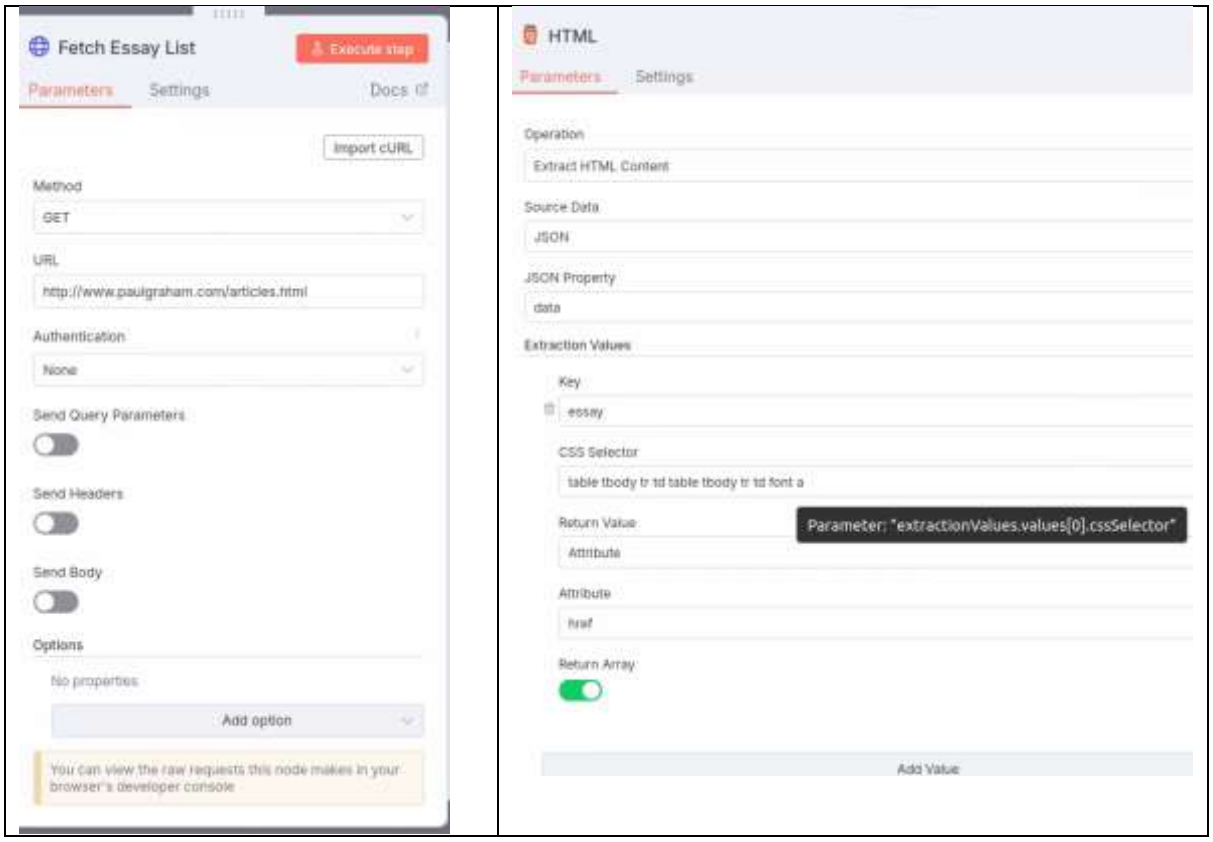
Pinecone Vector Store
When 'aiAgent' asks you a question regarding scores of students, you get the information from pine cone vector store

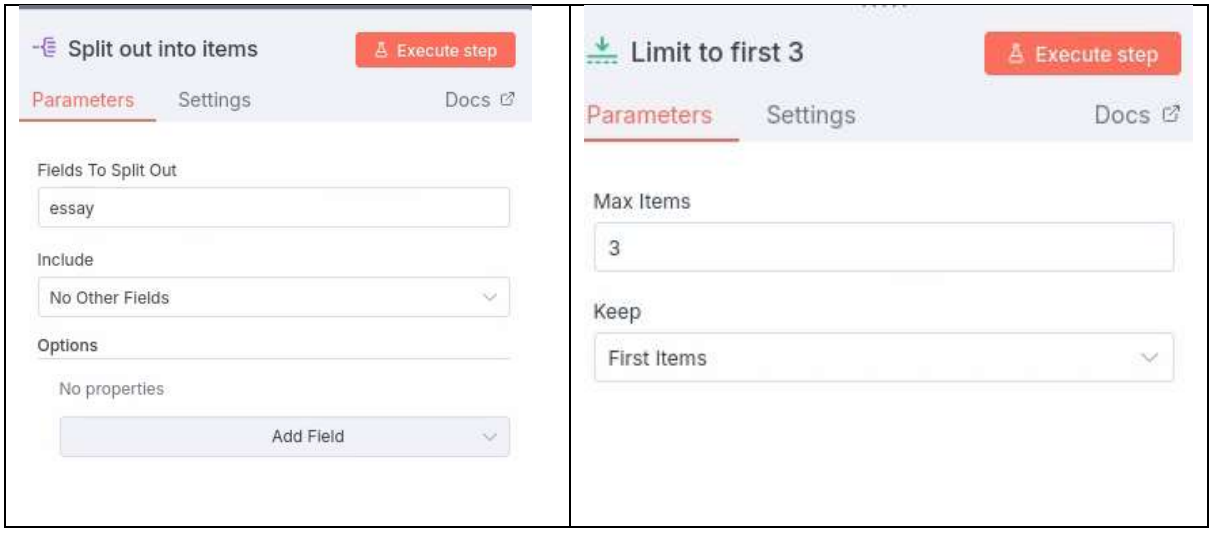
Z. Scrapping web-pages and summarization











n8n how to decide css selector in html node
Thinking
Kicking off 7 searches
Looking at 105 sites
Putting it all together

Deciding on the correct CSS selector in n8n's HTML node (formerly HTML Extract node) is crucial for accurately extracting the data you need from a web page. Here's a breakdown of how to approach this, including recommended practices and tools:
1. Using Browser Developer Tools (Inspect Element)
The easiest and most common way to identify the correct CSS selector is by using your web browser's developer tools.

- **Navigate to the web page:** Open the web page containing the data you want to scrape in your browser.
- **Inspect the element:** Right-click on the specific element (text, image, link, etc.) you want to extract and select "Inspect" or "Inspect Element". This will open the browser's developer tools and highlight the element's HTML code.
- **Copy the CSS Selector/Path:** In the developer tools (usually under the "Elements" tab), right-click on the highlighted HTML element and choose "Copy" > "Selector". This will copy the CSS selector to your clipboard

Using the CSS selector in the n8n HTML node

- **Add an HTML node to your workflow:** According to n8n Docs and, the node is called "HTML" (it replaced the older "HTML Extract" node).
- **Set Source Data:** Choose "JSON" if the HTML content is in the JSON output of a previous node (like an HTTP Request node), or "Binary" if the HTML is in a binary file.
- **Specify JSON Property (if applicable):** Enter the name of the JSON property containing the HTML.
- **Add Extraction Value:** Click "Add Value" to define what you want to extract.
- **Key:** Enter a name for the extracted value (e.g., "title", "price").
- **CSS Selector:** Paste the CSS selector you identified in the previous steps.
- **Return Value:** Choose the type of data to return: "Attribute" (e.g., "href" for links), "HTML", "Text", or "Value" (for input elements).
- **Return Array (if applicable):** If you expect multiple elements to match the selector, enable "Return Array" to get an array of values.

Example:
If you want to extract the text "Medium" from a `<span>` element within a table, the copied CSS selector might look like:

```
body > table:nth-child(2) > tbody:nth-child(1) > tr:nth-child(1) > td:nth-child(2) > span:nth-child(1)
```

Understanding CSS Selector Syntax:
Familiarize yourself with basic CSS selector syntax (e.g., `.` for classes, `#` for IDs, `>` for direct children, `nth-child()` for selecting elements based on their position) to create or modify selectors manually if needed.

AA.Send SMS using Twilio

1. Open Twilio home page:
2. Click on 'Start for free' button at the top-right
3. At the bottom of this page, click on Sign up with Google

4. After signing in, you have to enter your mobile number so as to verify the destination SMS address. Click the button *Send code via SMS*
5. You will receive a verification code on your mobile by SMS. Enter the code and click Verify button.
6. You will get a recovery code. Copy and Save it somewhere (maybe your email): XYTMNTY5PGJLS7FA52S5MY3YHP
7. Click *Continue* button
8. Fill up some survey information.
   Which channel are you interested in? ==> **Select SMS**
9. You are taken to following screen. This screen has two parts: Top and Bottom.

In the top part, you are invited to get a free telephone number from where you will get an SMS. Note down this number.

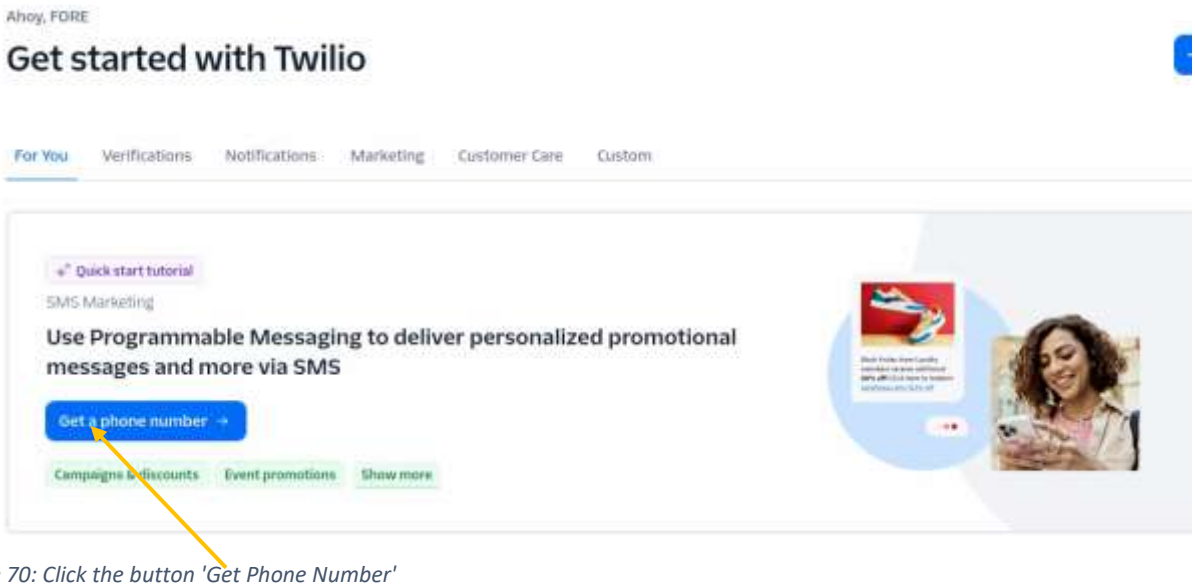


Figure 70: Click the button 'Get Phone Number'

In the bottom part of the page, you get *Account information* and *Auth token*. Save this information also.

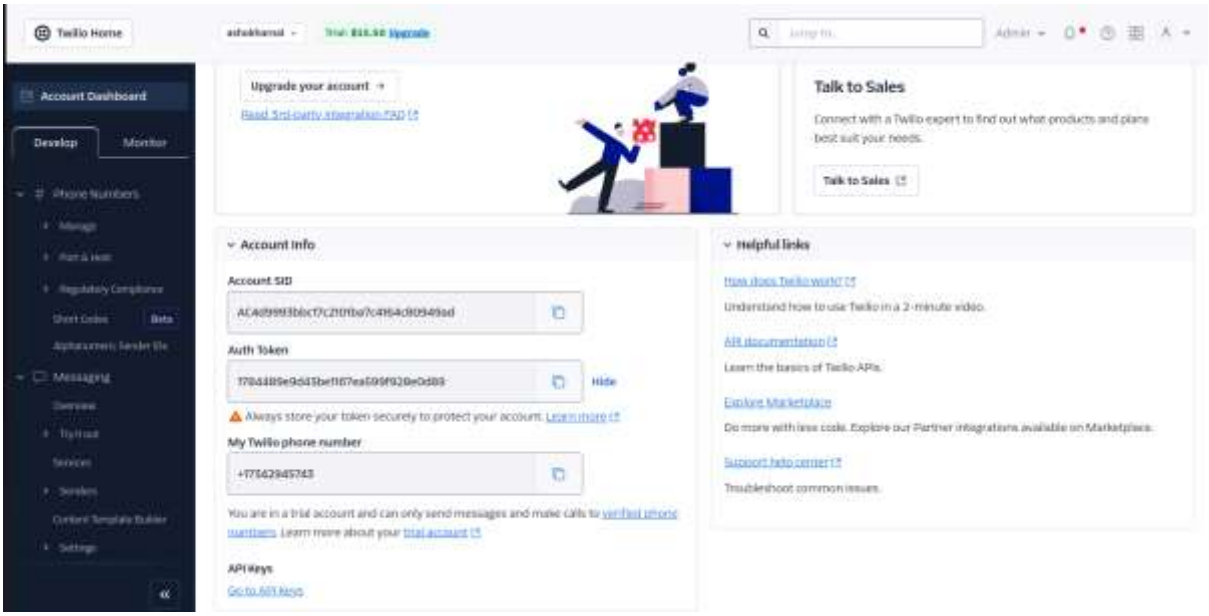


Figure 71: Copy each and every information from this page into your email

BB. Slack API

To work with Slack API, click on this link or in Google, search for Slack API. Click here OR here.

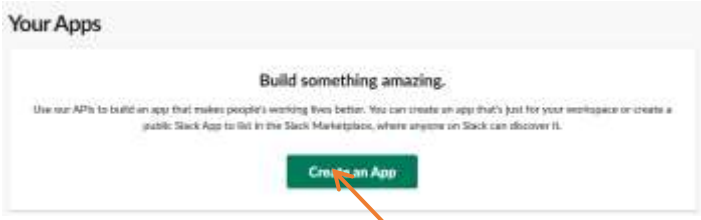


Figure 72: Click on Create an App

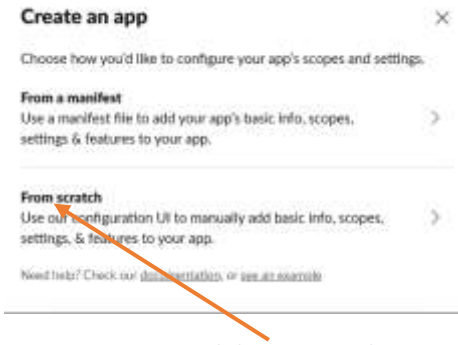


Figure 73: Click From Scratch

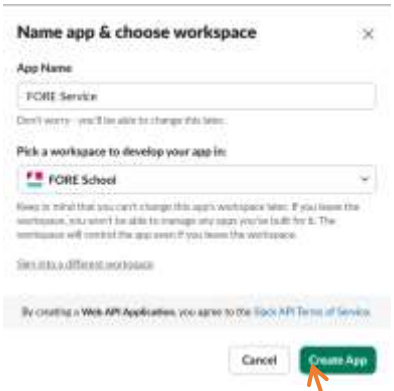


Figure 74: Fill it up and click Create App

Figure 75: Not sure if to note down these or not.

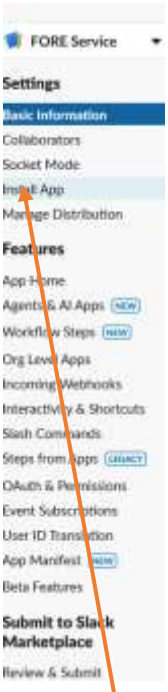


Figure 76: Click Install Apps

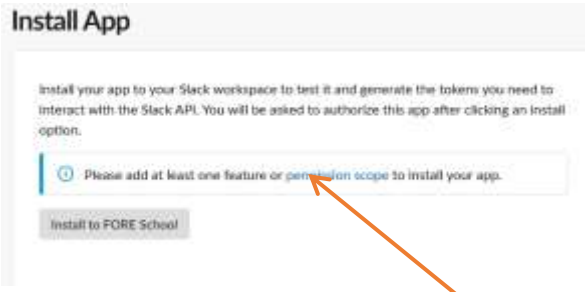


Figure 77: Click Install to FORE School. But first Give permissions

Figure 78: Allow your App permissions on your Workspace

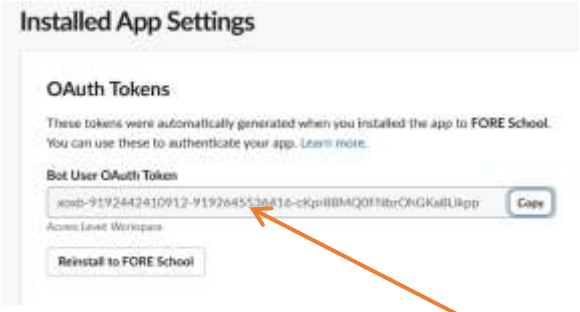
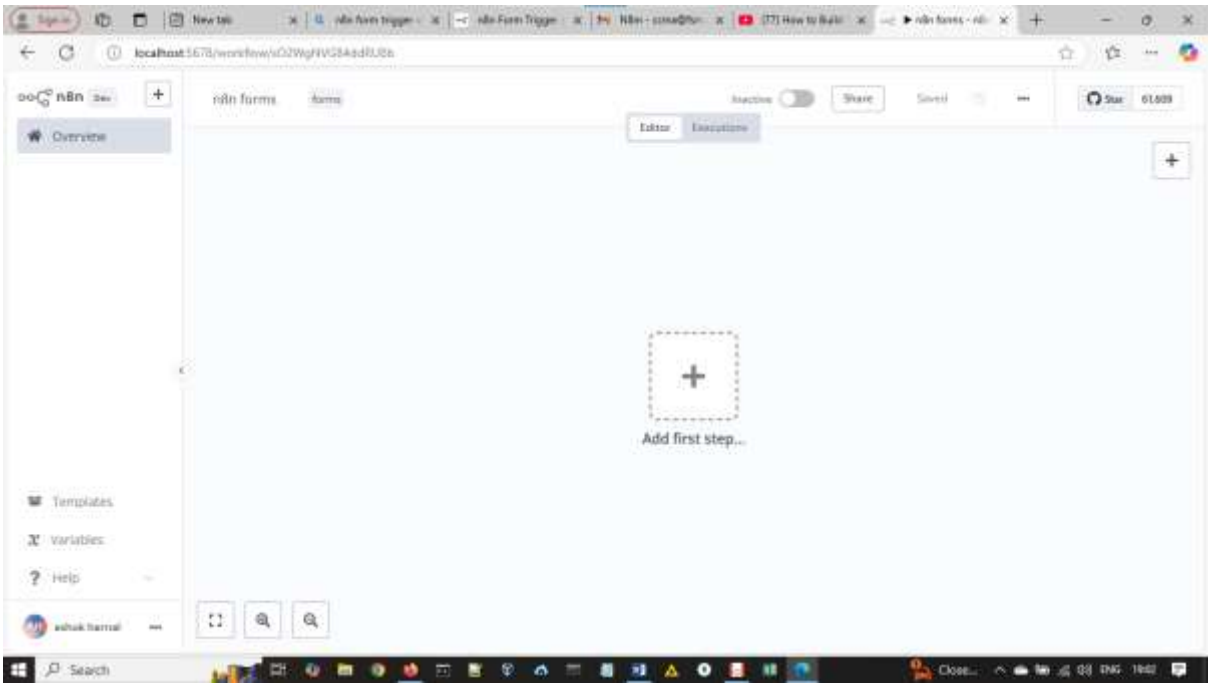


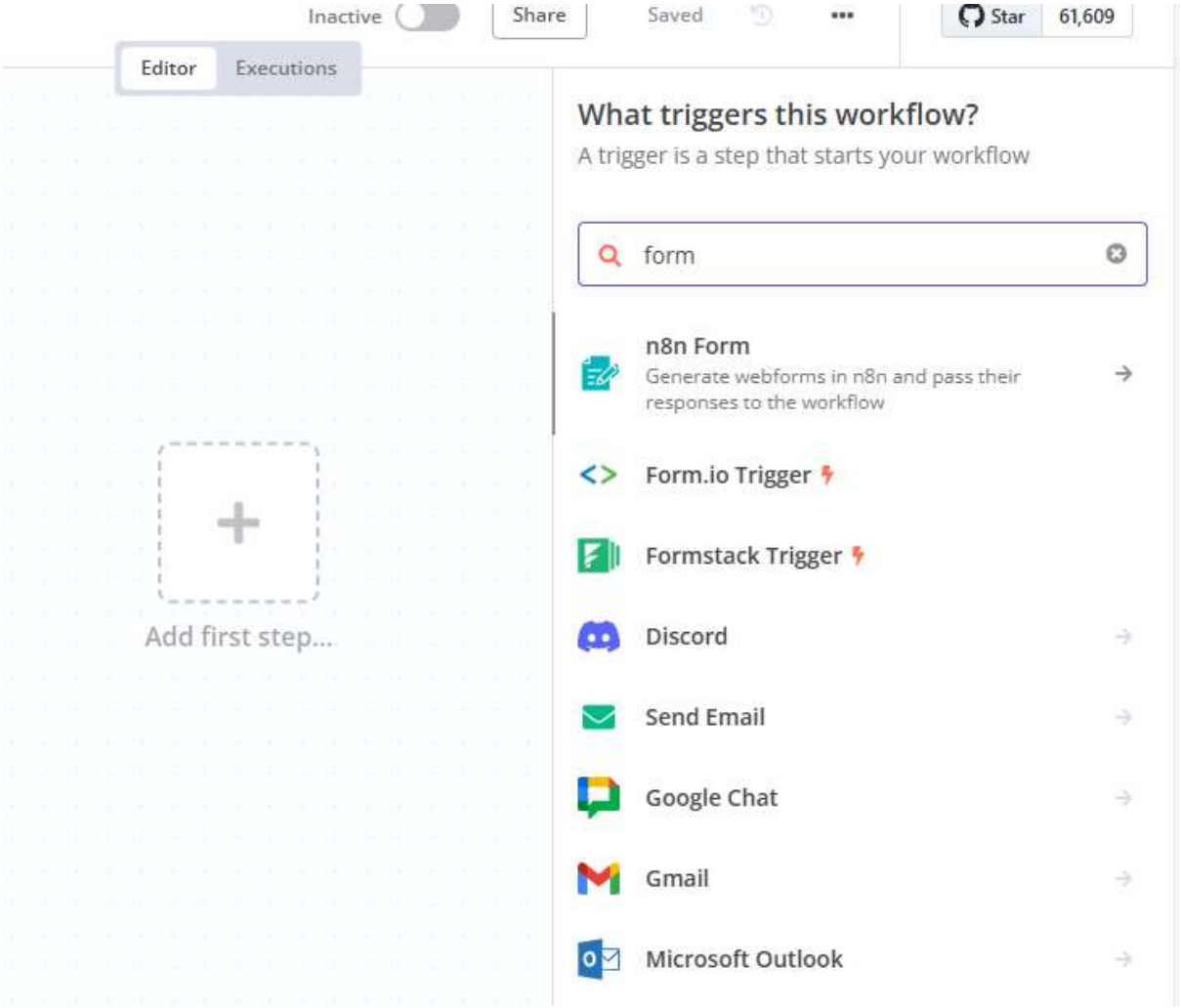
Figure 79: A token will be generated. Note this down. This token is important.

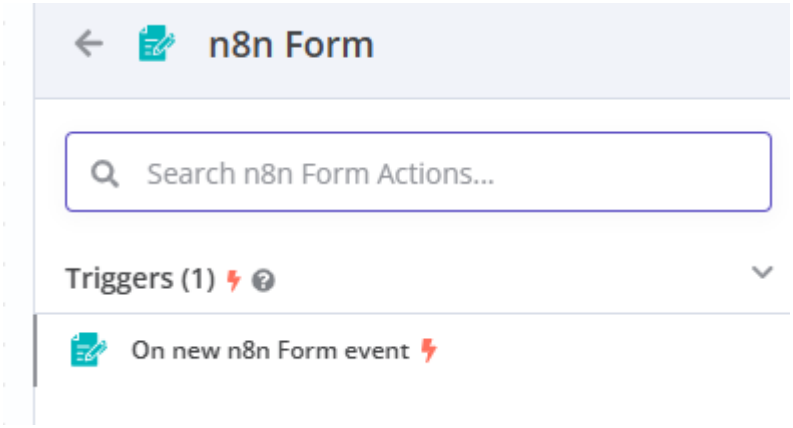
xoxb-9192442410912-9192645536416-cKpr88MQ0FNbrOhGKa8Llkpp

Figure 80: An App FORE Service is now visible

[YouTube video]







CC. Blog writer agent

See this video:

This blog writer searches a news on a topic given from the Internet using SerpAPI (Google Search) and then writes a blog. In the Blog Writer Agent, besides the System Message, also set source for Prompt (User Message) as below:

Figure 81: Blog Writer agent needs special configuration. See below.

TASK word as mentioned below should not be there.

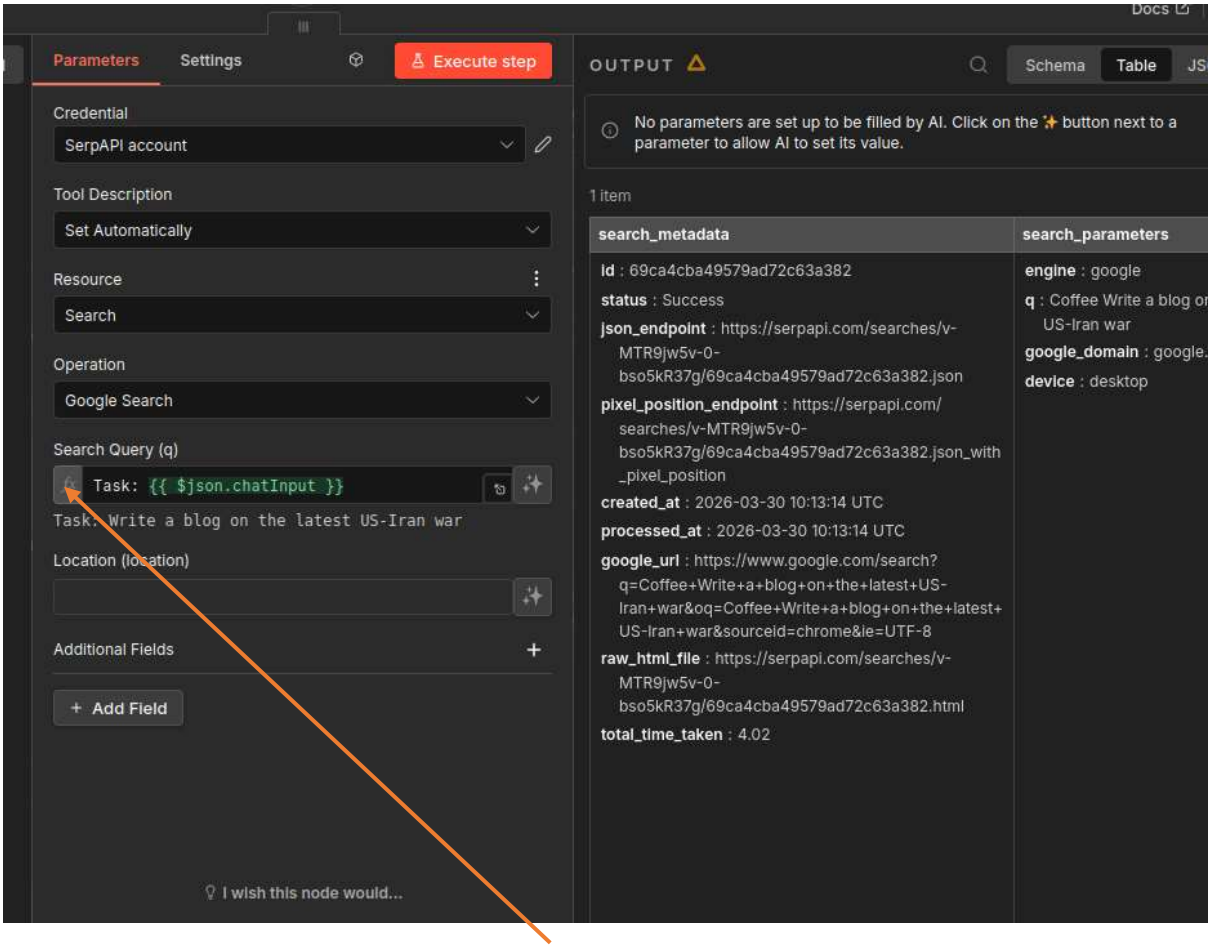


Figure 82: Official SerpAPI tool's configuration. REMOVE 'TASK' WORD. IT DOES NOT WORK WITH IT.

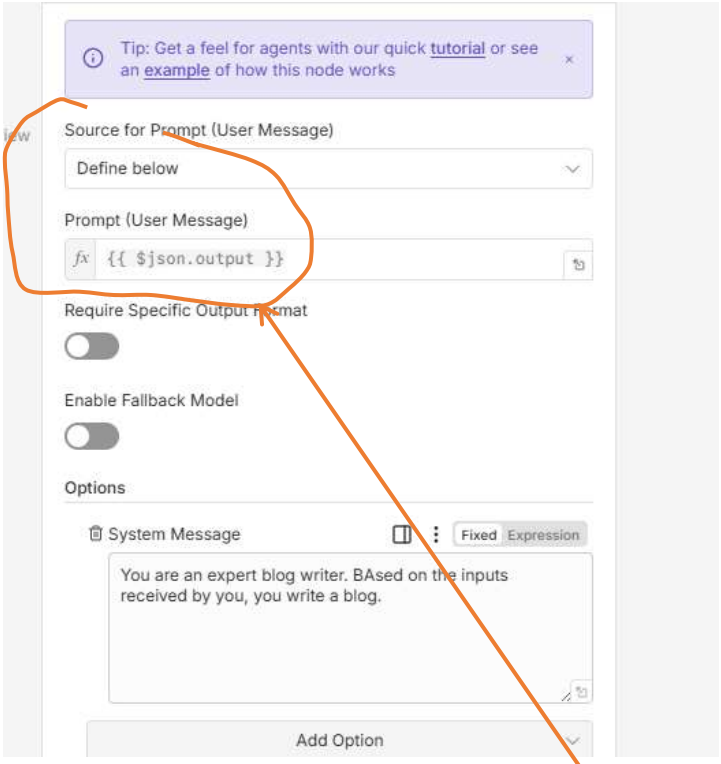


Figure 83: Blog Writer Agent: Besides System Message, make changes here.

DD. HTTP Request Node

There are two *HTTP Request* nodes. One pertains to HTTP method which has an output and the IInd is *HTTP Request Tool* which DOES not have any output—i.e. its input comes from AI agent and output goes to AI agent.

When to use HTTP Request Tool

Below example is for Http Request node BUT not tool with an output. Use *HTTP Request Tool* when you have a URL of say a FORM or a database from where certain values are extracted which the *AI Agent* can then use for further processing..

HTTP Request Method

HTTP request node can scrap data from given web-site. But some sites, such as Wikipedia, IMDB, forbid bots. Test the site: https://hackernoon.com/ . This site permits access. Fill up just the following in this node and click *Execute step* to see the output.

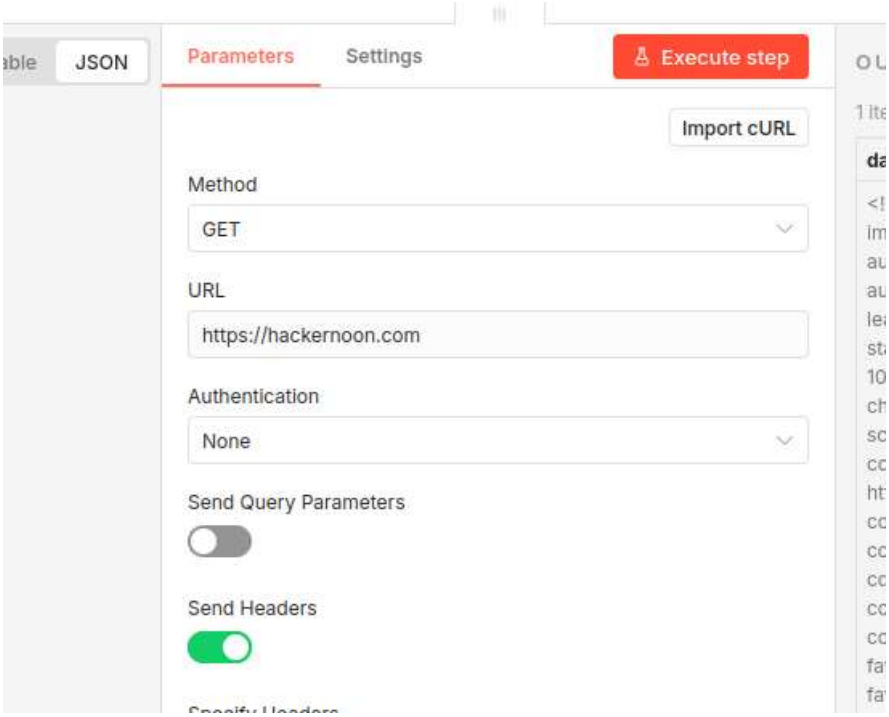


Figure 84: Only fill up the site URL to get the site

The extracted HTML page can be fed into an HTML node, from where required anchor or tag extracted as below:

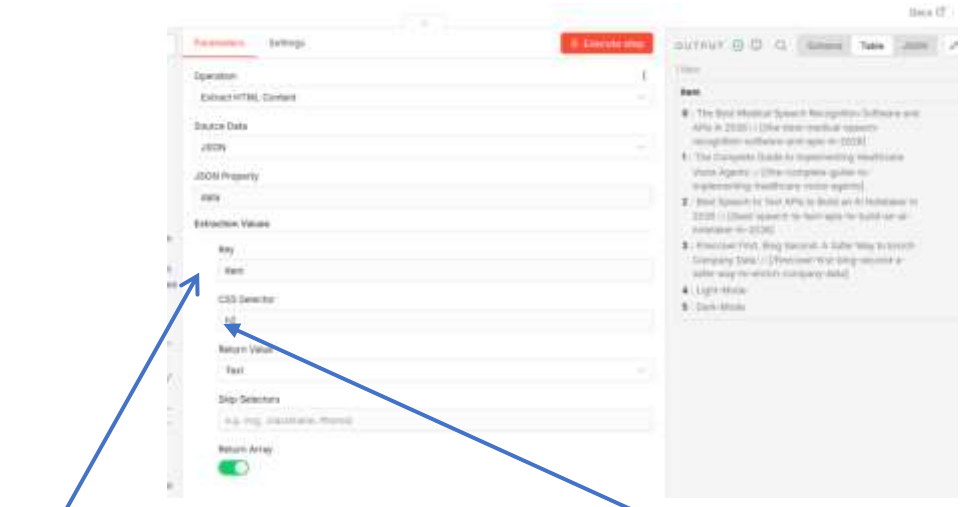


*Figure 85: **key** can have any name but test CSS Selector with values as: h1. h2, h3 or body WHICHEVER gives desired result*

Here is the full sequence:

Figure 86: HTTP request and HTML nodes are configured as above.

EE. Webhook demo-I

Folder: `C:\Users\ashok\OneDrive\Documents\n8n\13.webhook demo-II`
Files: `simple.html`
`webhook-demo-I.json`
For details about webhook, read demo-II below.

The following webhook workflow is triggered by a very simple html form (code written sideways).

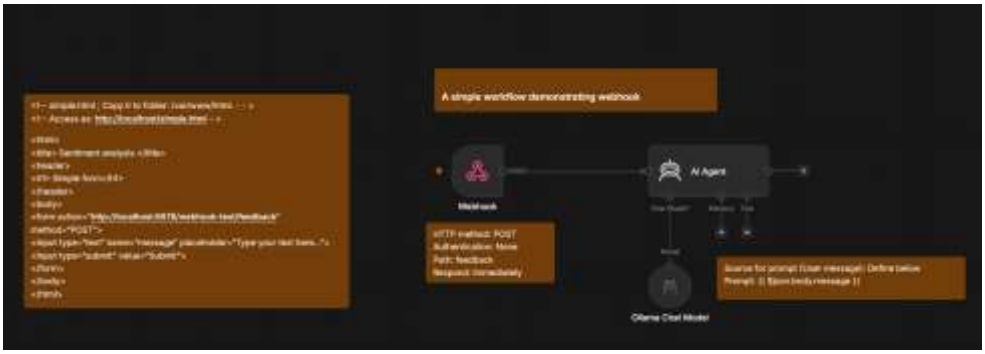


Figure 87: A simple webhook demo. IT is called by an html form on Apache webserver. Ollama chat model: llama3.2

FF. Webhook demo-II

Folder: `C:\Users\ashok\OneDrive\Documents\n8n\13.webhook demo-II`
Files:
`feeback.html`
`Simple-feedback.html`
`webhook.json`

A webhook essentially says: Here is my URL; call me back when an event happens and I will trigger further actions for you. A webhook is an automated, event-driven message sent between web applications. Instead of one app (App B below) constantly asking another (App A below) for updates (polling), the sending app (App A) instantly pushes data to a specific URL (endpoint) the moment a defined event occurs.

The Event: An action occurs in App A (e.g., a customer makes a payment or submits a form).
The Payload: App A gathers the relevant data about that event (usually in JSON format).
The Delivery: App A fires an HTTP POST request, sending that payload directly to a specific URL provided by App B. App B has a webhook.

In the context of webhooks, a **callback** (specifically an "HTTP callback") is the exact **URL or destination address** that you provide to the sending application (App A) so it knows where to deliver the data.

When you configure a webhook, you are essentially telling the App A application: *"When this event happens, call me (i.e. App B) back at this specific web address."*

Figure 88: The flow. Call Ollama is HTTP Request node

Here are the configurations:

My prompt on Claude Desktop:

I am working on my Intranet. I have n8n installed on my laptop. I also have ollama installed as also Apache webserver and PostgreSQL installed. I want to design a simple n8n experiment to demonstrate the webhook usage. I do not have api access to Gmail or google sheets or to other google products.

Response:

This is a great use of what you already have — you can build a fully local, self-contained webhook demo without touching any Google service. Here's a clean design that exercises all four tools together: Apache serves a simple form, n8n's Webhook node receives the submission, Ollama analyses the text, and PostgreSQL stores the result.

The idea

An HTML feedback form (served by Apache) posts text to an **n8n Webhook**. n8n forwards that text to **Ollama** for analysis (e.g. sentiment or a one-line summary), writes the original text plus Ollama's response into **PostgreSQL**, and then replies to the original request with a JSON result. Every piece of software you already have gets used, and nothing leaves your laptop.

1. Postgres: create a table

```
CREATE TABLE feedback_log (
    id SERIAL PRIMARY KEY,
    submitted_text TEXT NOT NULL,
    ollama_response TEXT,
    created_at TIMESTAMP DEFAULT NOW()
);
```

Proceed as follow to create table in database 'harnal', user: Harnal, password: harnal:

Figure 89: Create table now...

2. n8n workflow

Node 1 — Webhook

- HTTP Method: POST
- Path: feedback
- Response Mode: Using *'Respond to Webhook'* Node

This gives you a production URL like http://localhost:5678/webhook/feedback OR a test URL like: http://localhost:5678/webhook-test/feedback

Node 2 — HTTP Request (call Ollama)

- Method: POST
- URL: http://localhost:11434/api/generate
- Body (JSON):

```
{
  "model": "llama3.2",
  "prompt": "In one short sentence, describe the sentiment of this feedback: {{ $json.body.text }}",
  "stream": false
}
```

(swap llama3.2 for whatever model you've pulled in Ollama — check with ollama list)

Node 3 — Postgres (Insert)

- Operation: Insert
- Table: feedback_log
- Columns:
  - submitted_text → {{ $('webhook').item.json.body.text }}
  - ollama_response → {{ $json.response }}

Node 4 — Respond to Webhook

- Response Body:

```
{
  "status": "ok",
  "analysis": "{{ $('HTTP Request').item.json.response }}"
}
```

3. Apache: the form

A tiny static page (`feedback.html`) served by Apache that posts to the n8n webhook with fetch:

```
$ sudo cp feedback.html /var/www/html/
```

```
<!DOCTYPE html>
<html>
<body>
  <h3>Leave feedback</h3>
  <textarea id="text" rows="4" cols="40"></textarea><br>
  <button onclick="submitFeedback()">Submit</button>
  <p id="result"></p>

  <script>
    async function submitFeedback() {
      const text = document.getElementById('text').value;
      const res = await fetch('http://localhost:5678/webhook/feedback', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ text })
      });
      const data = await res.json();
      document.getElementById('result').innerText = data.analysis;
    }
  </script>
</body>
</html>
```

Drop this in Apache's document root (e.g. /var/www/html/feedback.html), and it's live on your intranet and access as: http://localhost/feedback.html .

If you are having *Test-URL* and not *production* URL for webhook, modify one line within the script as:

```
const res = await fetch('http://localhost:5678/webhook-test/feedback',
```

4. Test without the form first

Before wiring up Apache, just hit the webhook directly write in Ubuntu terminal and press ENTER to confirm that the n8n side works:

```
curl -X POST http://localhost:5678/webhook/feedback \
  -H "Content-Type: application/json" \
  -d '{"text": "The new intranet page is really slow to load."}'
```

You should get back JSON with Ollama's take on the sentiment, and a new row in feedback_log.

4. What does the function inside the script do?

This function runs when someone clicks "Submit" on the feedback form, and it walks through the whole round trip to your n8n workflow. Step by step:

1. **const text = document.getElementById('text').value;** Grabs whatever the user typed into the <textarea id="text"> on the page.
2. **const res = await fetch('http://localhost:5678/webhook/feedback', {...})** Sends that text as an HTTP POST request to your n8n webhook. The await pauses this function until n8n finishes processing and sends back a response — so it waits through the Ollama call and the Postgres insert before continuing.
    - method: 'POST' — this is a POST request, matching the Webhook node's configured method.
    - headers: { 'Content-Type': 'application/json' } — tells n8n the body is JSON, not plain text or a form.
    - body: JSON.stringify({ text }) — packages the text into a JSON object like {"text": "the page is slow"} and converts it to a string, since fetch bodies have to be strings, not raw objects.
3. **const data = await res.json();** Takes the response n8n sends back (from your Respond to Webhook node, which returns something like {"status": "ok", "analysis": "..."}) and parses it from a JSON string into a JavaScript object.
4. **document.getElementById('result').innerText = data.analysis;** Pulls the analysis field out of that object — Ollama's sentiment description — and writes it into the <p id="result"> element on the page, so the user sees the result without the page reloading.

One gap worth flagging: there's no error handling. If the webhook is down, returns a non-200 status, or data.analysis is undefined for some reason, this will either throw an unhandled error or silently write "undefined" into the result box..

If you're fine with the page navigating (or reloading) after submit, you don't need JS or fetch:

```
$ sudo cp simple-feedback.html /var/www/html/
```

```
<form action="http://localhost:5678/webhook/feedback" method="POST">
  <textarea name="text" rows="4" cols="40"></textarea>
  <button type="submit">Submit</button>
</form>
```

The browser handles the POST itself. The catch: it'll navigate to the webhook URL and show n8n's raw JSON response as the new page, unless you set the form's target to a hidden <iframe> to suppress that.

Flow architecture

The full architecture. Note the loop of response.

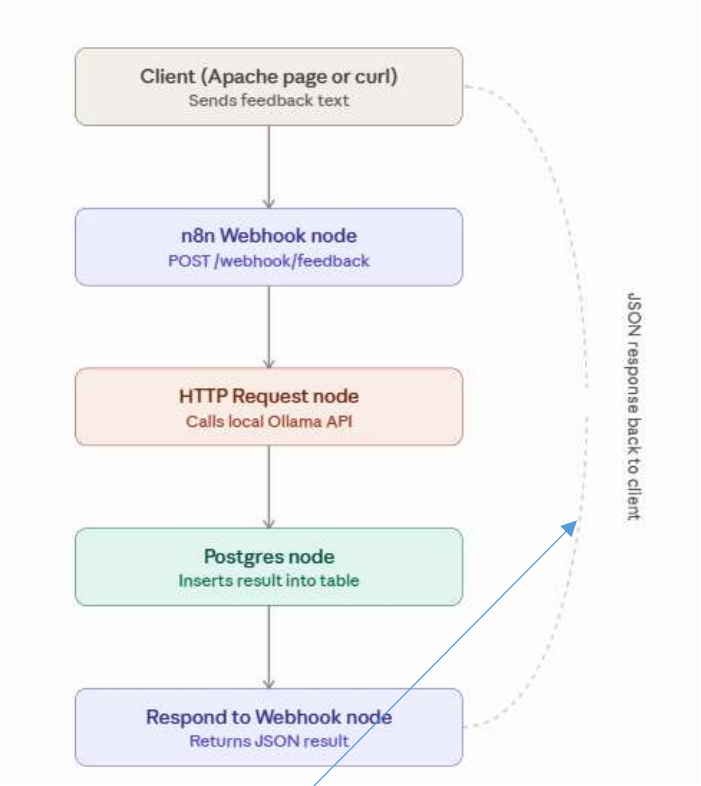


Figure 90: Note the response back to client

The path: you typed *feedback* into the Webhook node, which is why the URL ends in */webhook/feedback*. If you'd typed *submit-form* instead, that would be the URL.

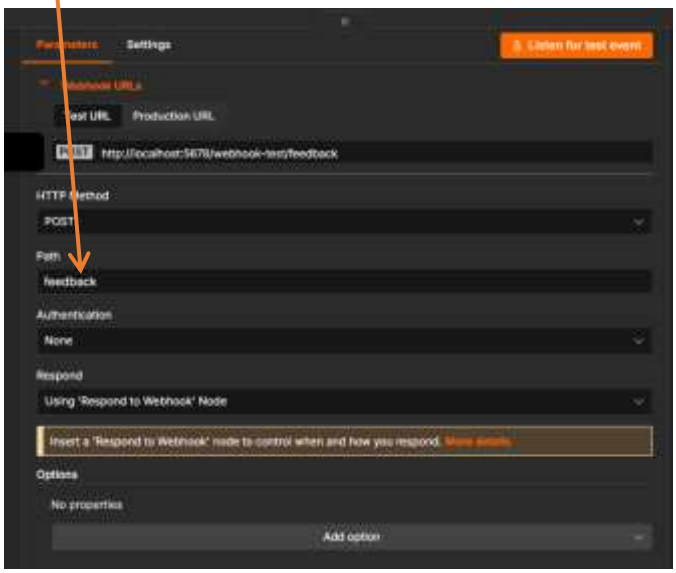


Figure 91: webhook node configuration.
Note: it is 'webhook-test/feedback' AND not 'webhook/feedback'

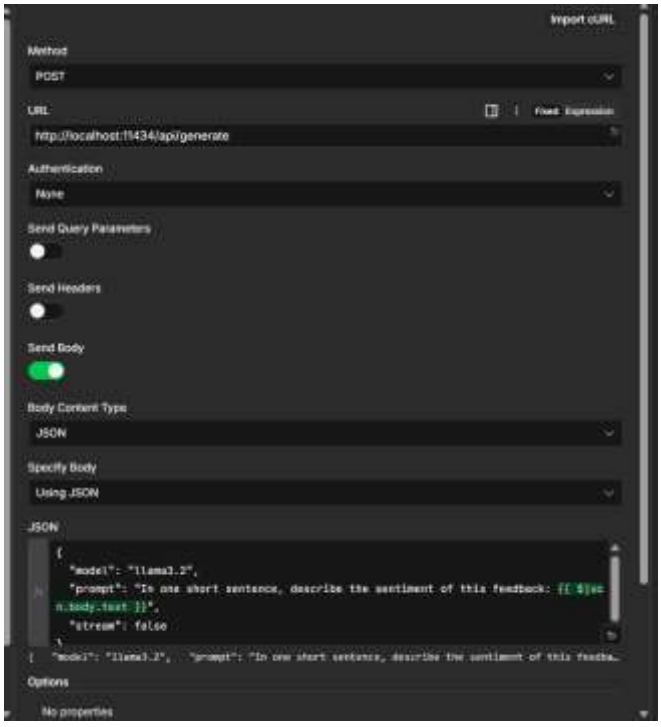


Figure 92: HTTP Request node calling ollama URL

Figure 93: Postgres node

Figure 94: Respond to webhook node

In n8n, a node **does not automatically receive the outputs of all previous nodes** in the workflow. Instead, a node only receives the input from the immediately preceding node to which it is connected. If you need data from a node that is not immediately before the current node, you can use **expressions** to reference it specifically by its node name (as above):

- **Syntax:** Use `$( 'Node Name' ).item.json` to pull data from a specific previous step.
- **All Data:** Use `$( 'Node Name' ).all()` to access all items from that node.

GG. DataTable

Also refer this link

Nodes used (in sequence):

a. Form node: Collects: Name, email, Question, Answer
b. If node: Checks if email is from domain: *fsm.ac.in*
c. Edit node or Set node: Appends a field: *isTrusted* to input data received from the form. If email contains domain *fsm.ac.in*, value of *isTrusted* is true else it is false.
d. No Op node: It merely forwards its input to output. *No Op* node has two inputs. Out of the two inputs, only one of them would be having some data. So, the *No Op* node forwards that input only.
e. Basic LLM Chain: It considers two inputs: Question and Answer and generates relevant tags.
f. DataTable→ A Data table must pre-exist containing five columns by ANY name: Name, email, Question, Answer and tags. Form data fields, *isTrusted* field and tags get inserted as one row.

Figure 95: isTrusted node on true branch of if, appends a column isTrusted with value true. On the false branch value added is false.

Our Datatable name is *Data*. The DataTable panel is as below:

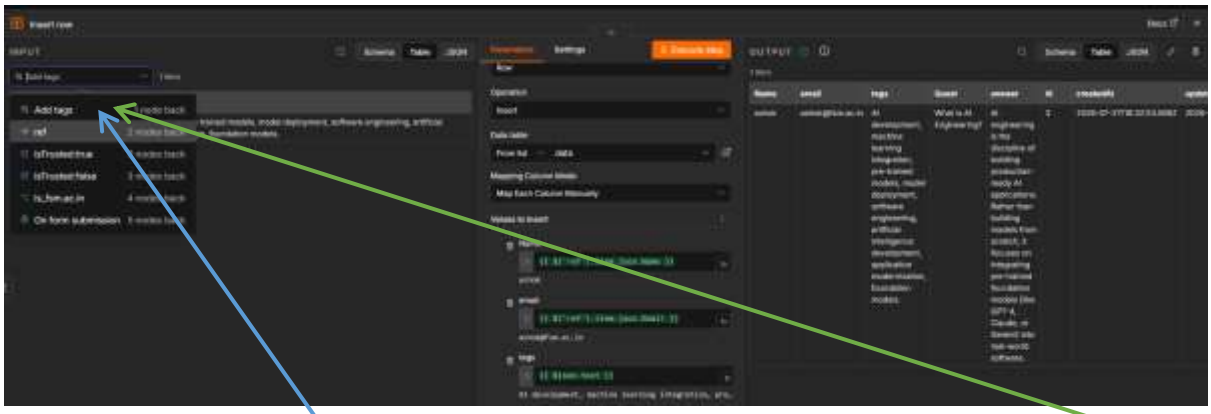


Figure 96: Columns in DataTable can receive inputs not only from the Basic LLM Chain but also from anyone of the previous nodes. (See below the expanded view).

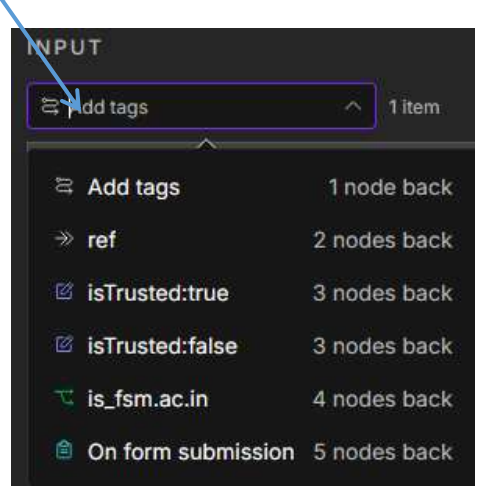


Figure 97: Input to DataTable columns can be from any one of the above nodes.

See DataTable configuration below.

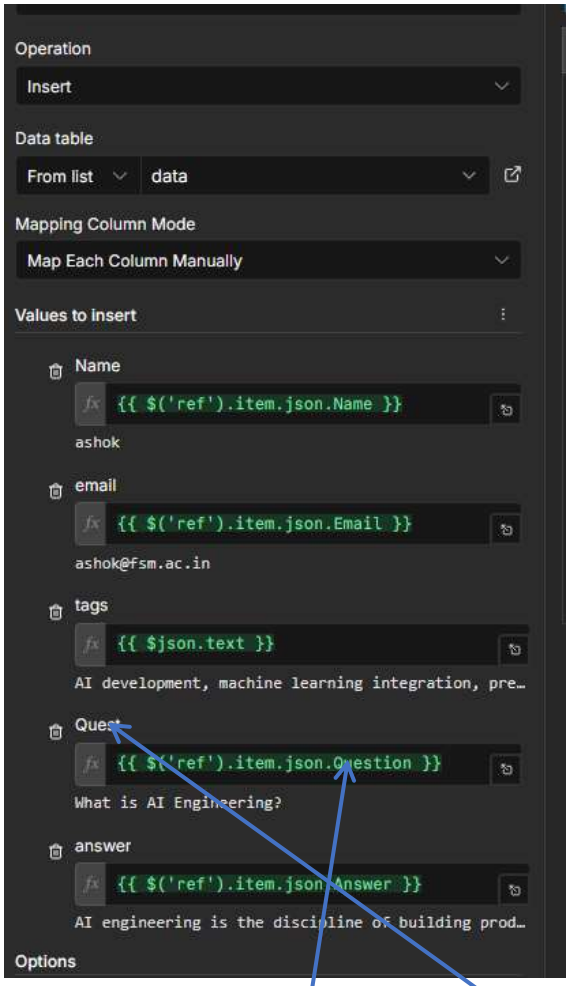


Figure 98: DataTable name is Data*. Note that one column name in DataTable is 'Quest' but form has output label as 'Question'.*

##########